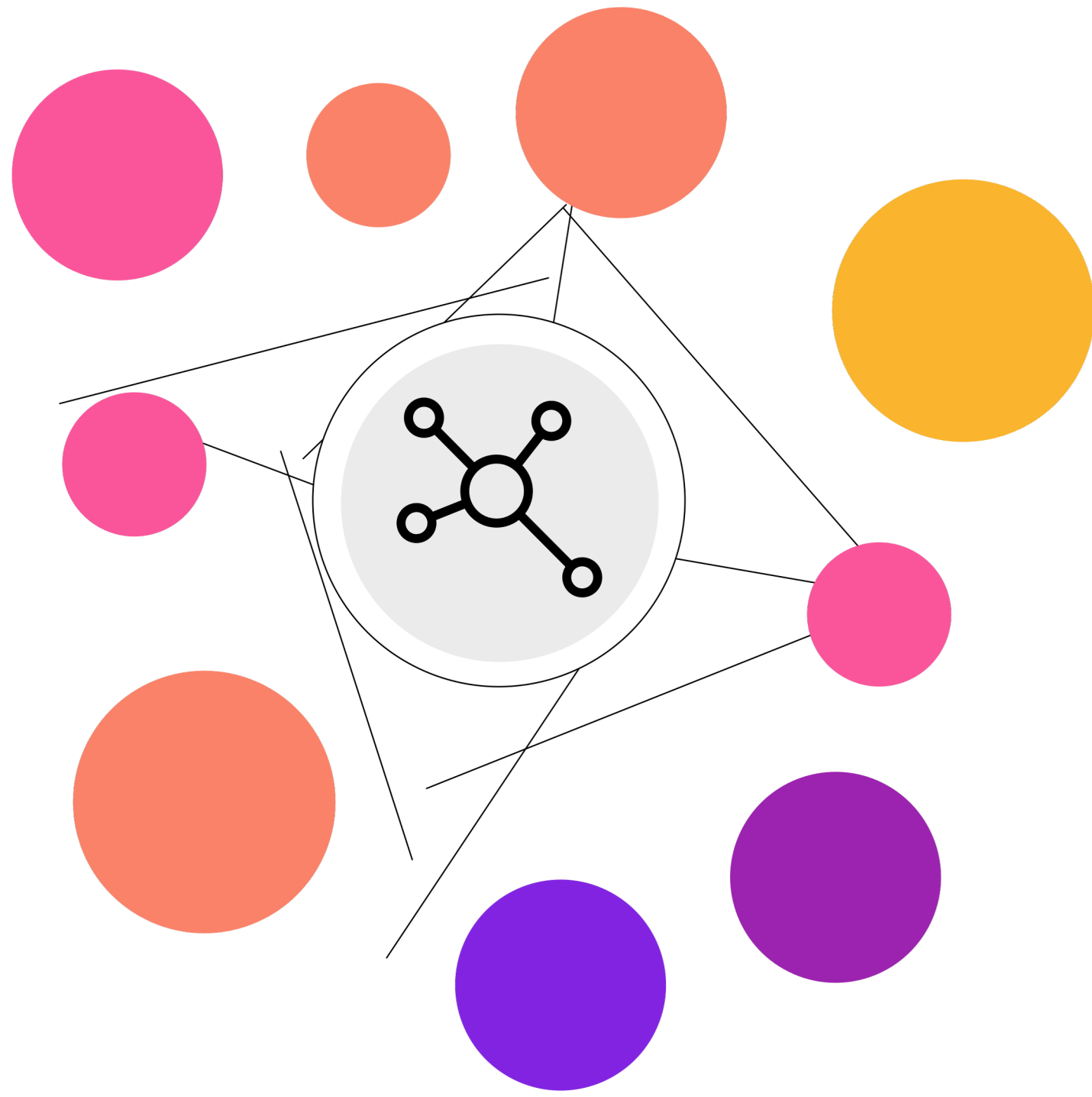


CS328



Learning Representations for Graph Retrieval

Omm Arindam (24110229)

Priyanshu Raj (24110279)

Ramji Purwar (24110287)

Context and Problem Statement



Data

Real-world systems can be represented as graphs capturing relationships between entities. Examples include molecules, social networks, and knowledge graphs.



Communities

The task is to measure structural similarity between graphs.
This depends on topology, connectivity patterns, and node/edge features.



Relevance

Graph similarity is crucial for applications like drug discovery and recommendations.
It enables retrieval of relevant graphs from large datasets.



Complexity

Exact similarity measures like GED are NP-hard.
Brute-force comparison across large corpora is computationally infeasible.

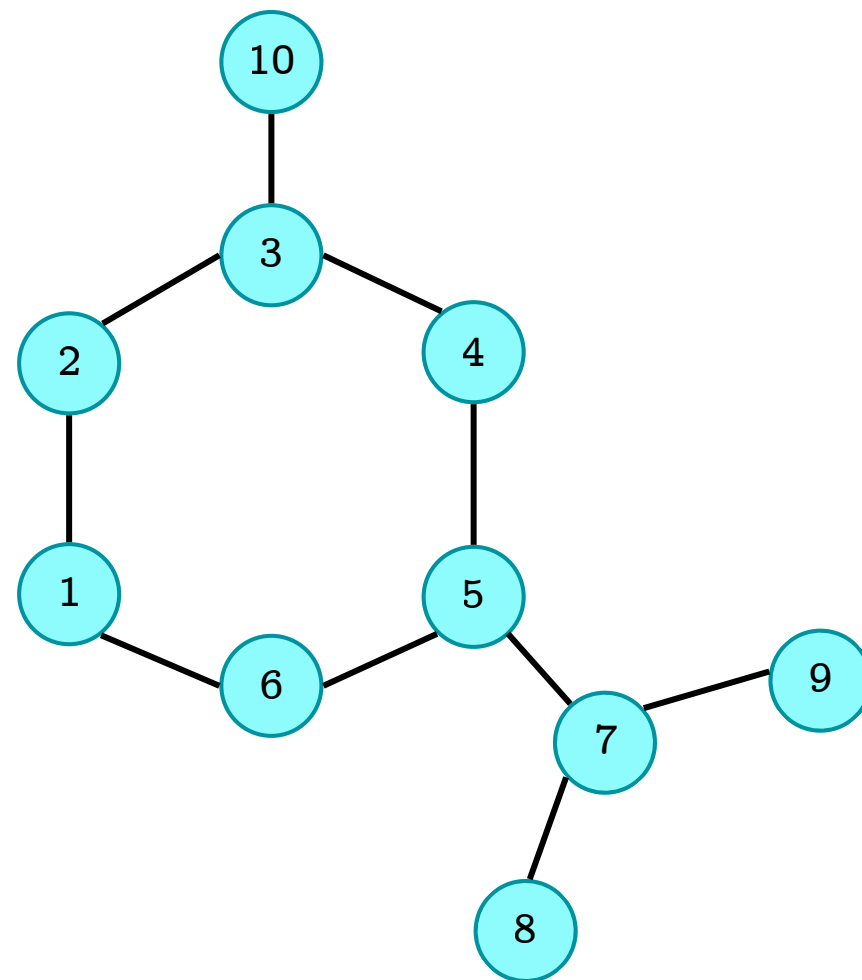
Broad Problem Statement:

Given a corpus $C = \{G_1, G_2, \dots, G_n\}$ and a query graph G_q ,
retrieve the top k-most similar graphs without exhaustive comparison.

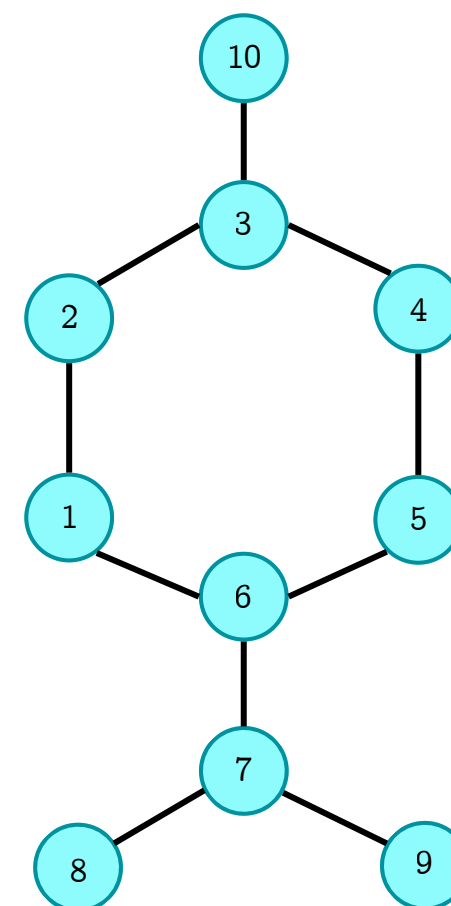
Context and Problem Statement

Given a corpus $C = \{G_1, G_2, \dots, G_n\}$ and a query graph G_q ,
retrieve the top k-most similar graphs without exhaustive comparison.

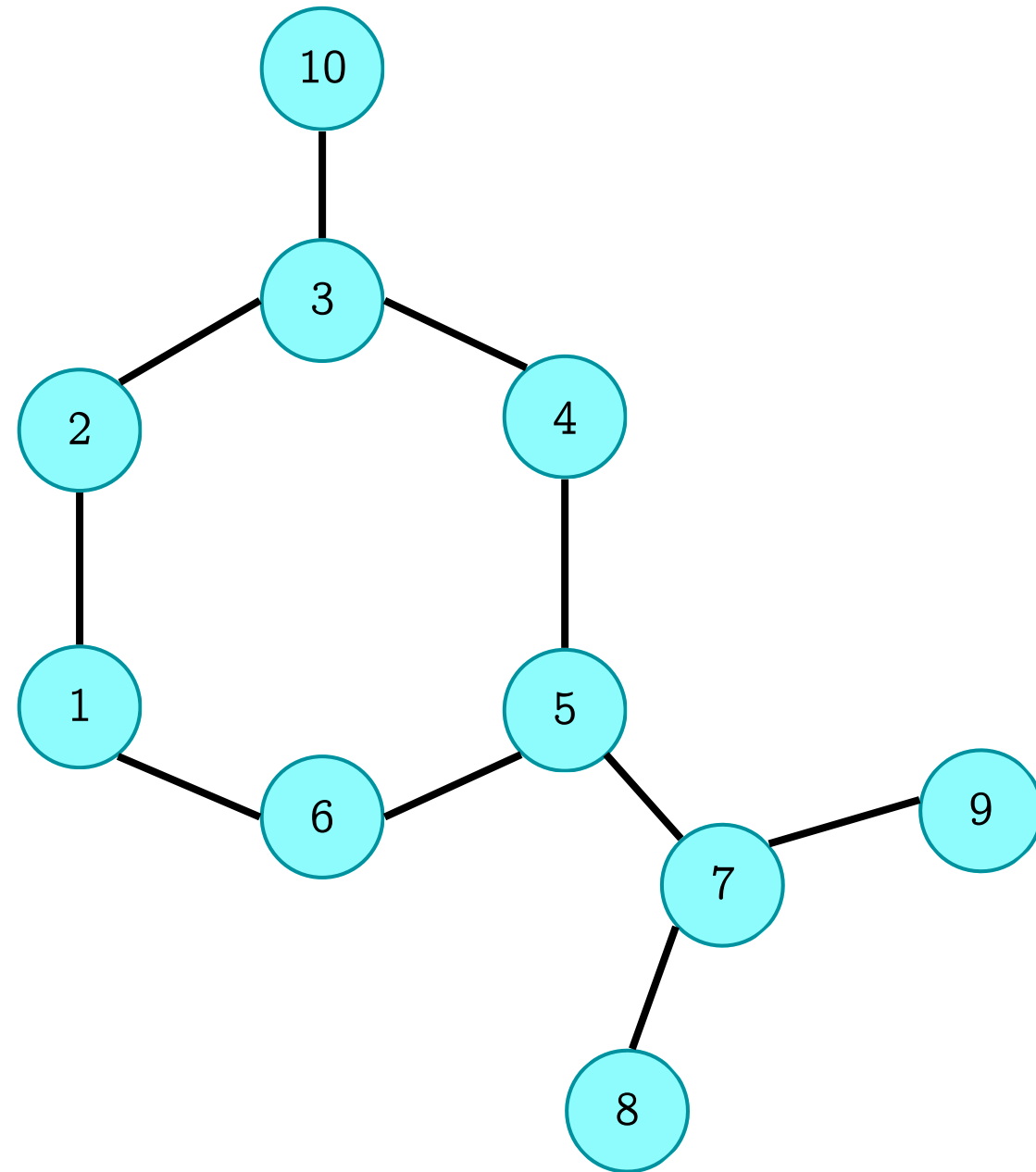
Structural similarity is measured using Graph Edit Distance (GED), which computes minimum number of operations needed to transform one graph to another.



similar to



Graph Similarity via GED



Graph A

Graph Edit Distance (GED)

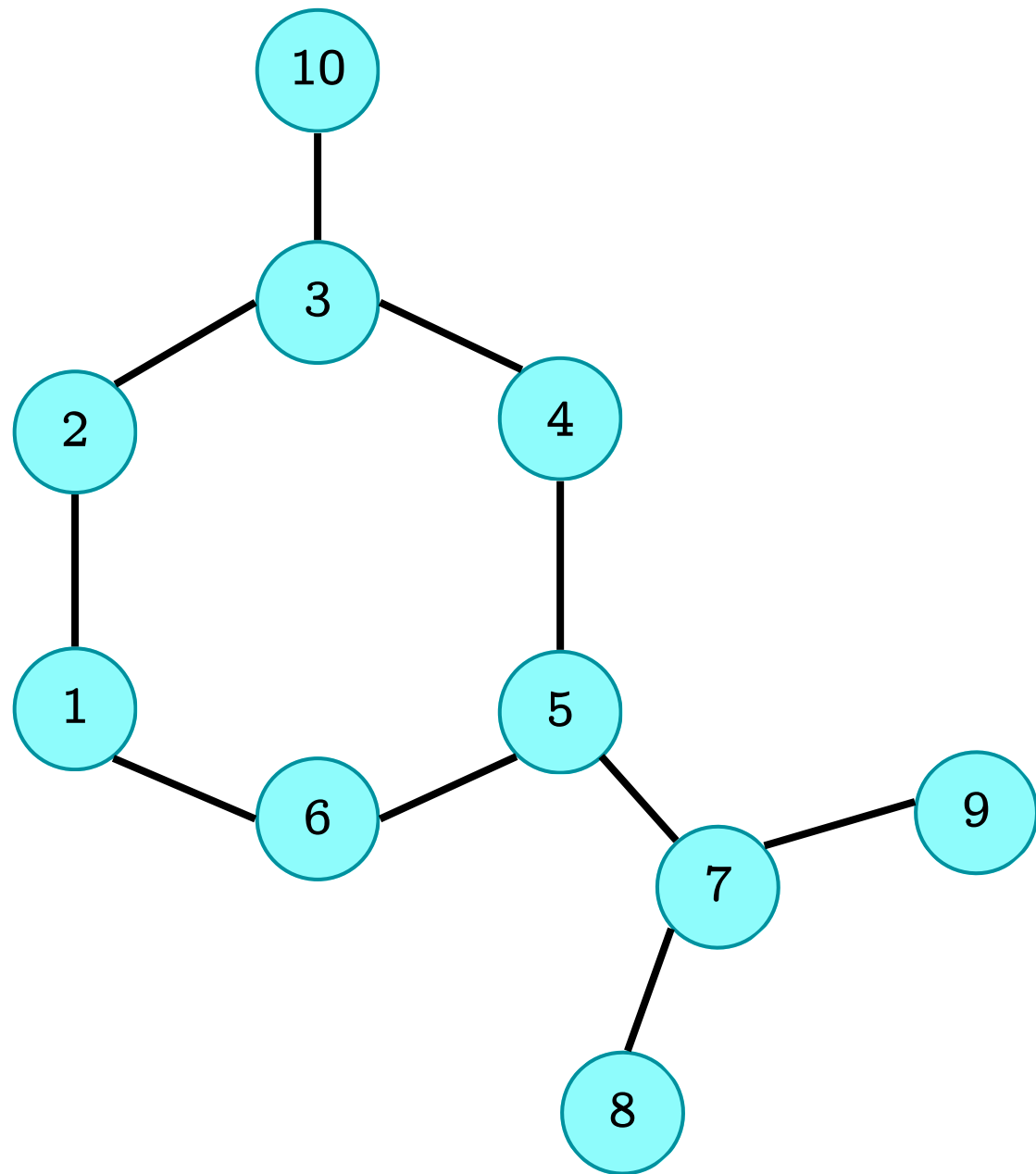
Measures similarity as the minimum number of operations required to transform one graph into another.

Allowed Operations

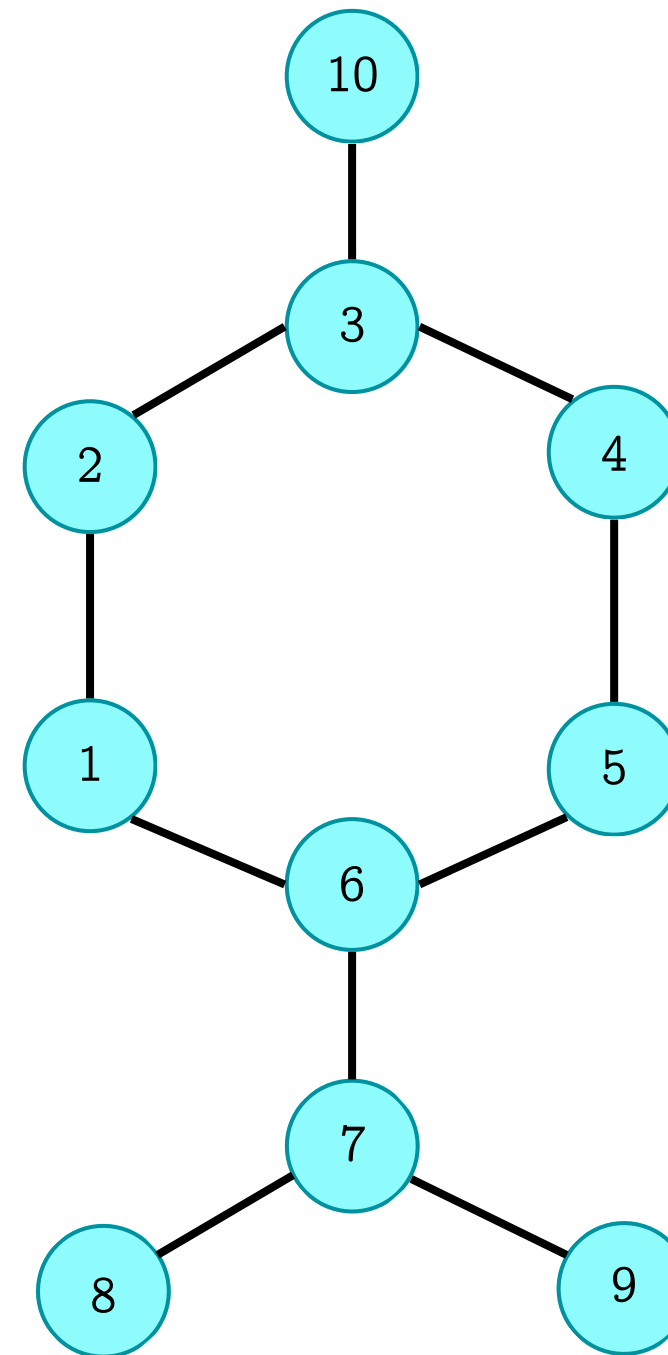
- Node insertion / deletion
- Edge insertion / deletion
- Node/edge substitution

Smaller GED $\Rightarrow$ graphs are more similar

Graph Similarity via GED

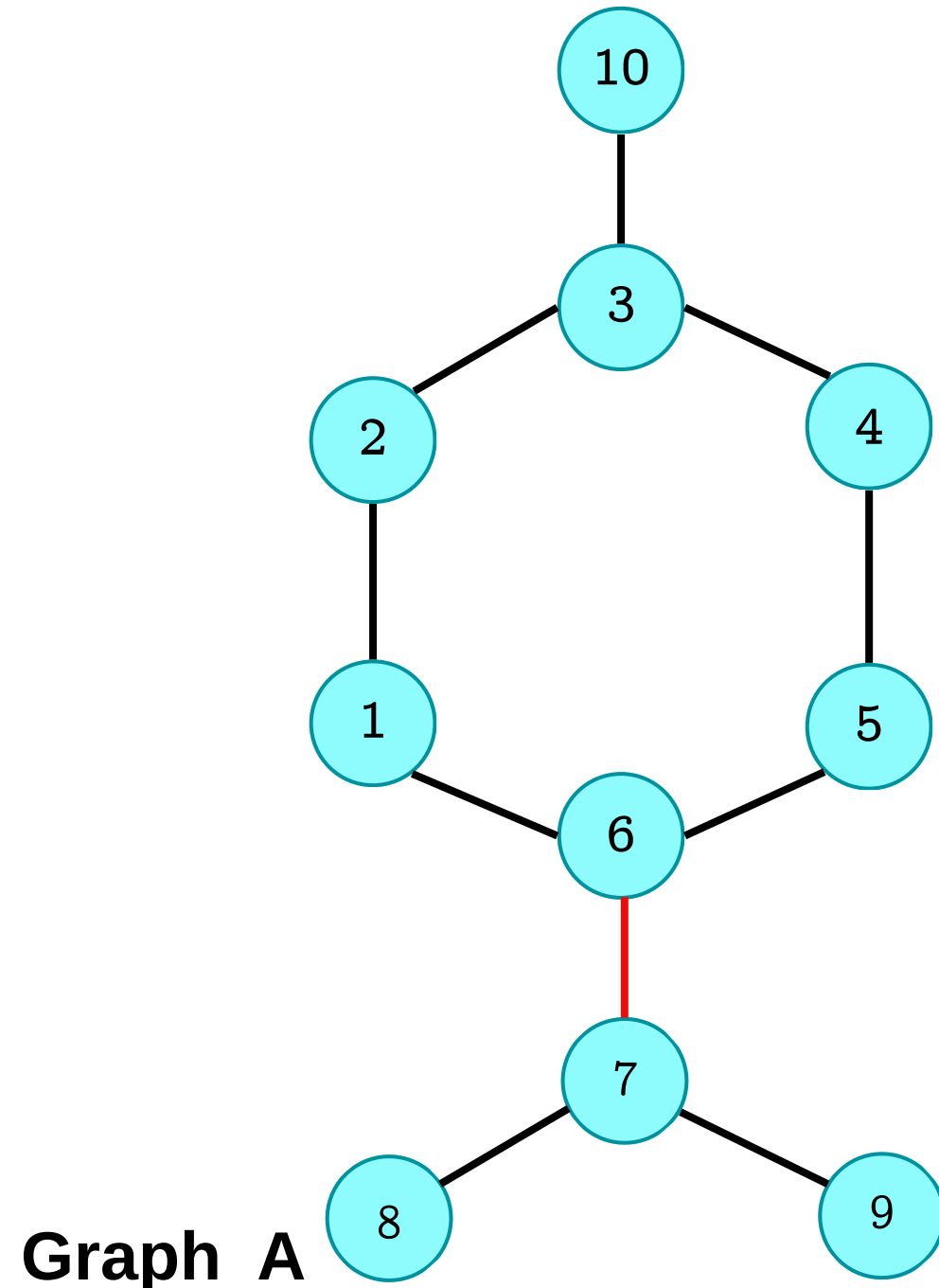


Graph A



Graph B

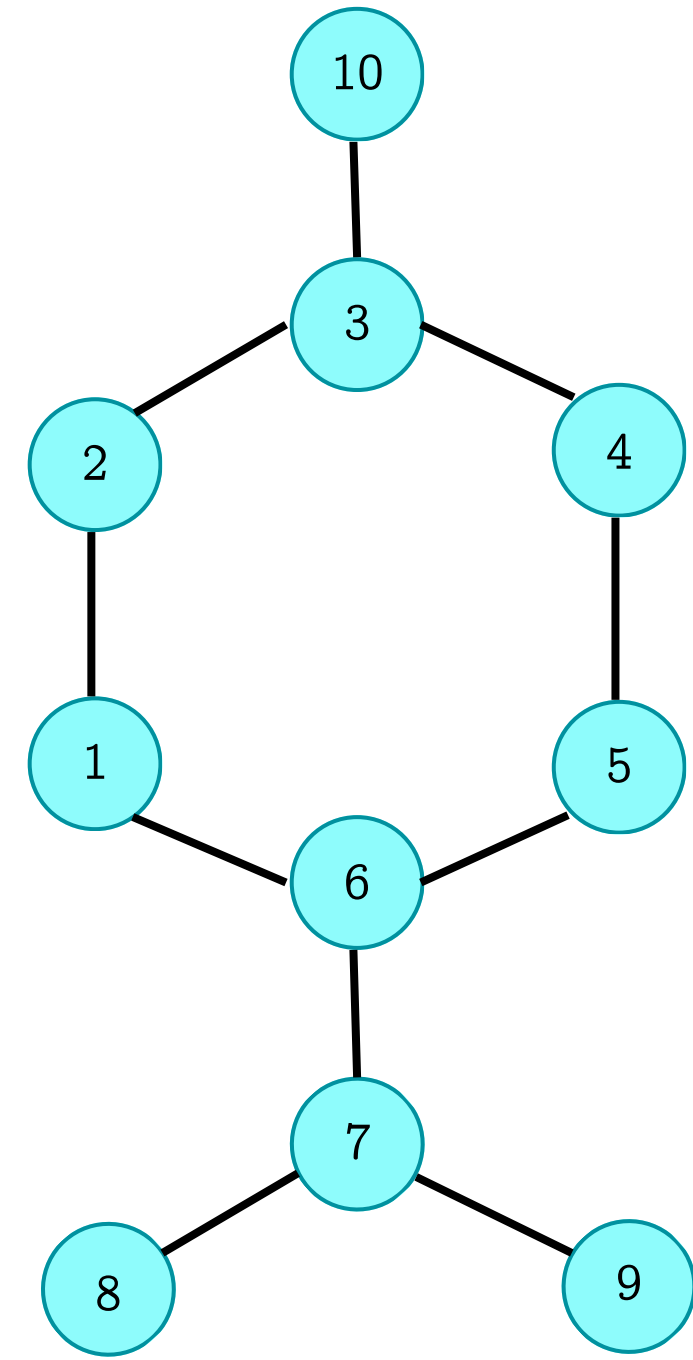
Graph Similarity via GED



Remove edge btw 5 and 7
Add Edge between 6 and 7
op_count = 2

=

GED = 2

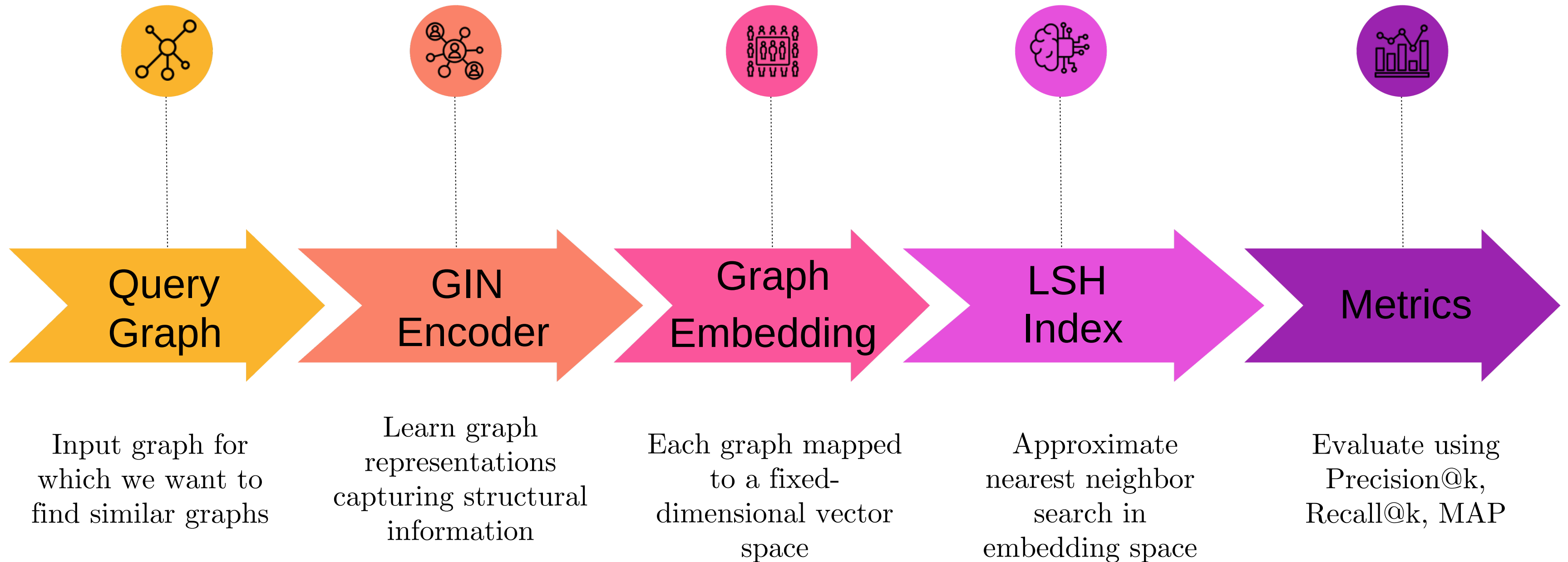


Limitations of using GED

Limitations of using GED

**Computing GED is NP-hard,
making it impractical for large datasets.**

Retrieval Pipeline



Datasets Used

Datasets	Domain	Graphs	Avg Nodes	Node Features
MUTAG	Molecular	188	17.9	Atom type (7-dim)
PROTEIN	Biological	1113	39.1	Structure (3-dim)
AIDS	Molecular	2000	15.6	Atom type (38-dim)
IMDB-Binary	Social	2000	19.8	Degree (31-dim cap)
Reddit-Binary	Social	2000	429.6	Degree (11-dim cap)

Graph Embedding using GIN

We use a Graph Isomorphism Network (GIN) to learn graph-level embeddings that capture structural similarity between graphs.

Graph Representation

- Node features updated using neighborhood aggregation
- Graph embedding obtained via Global Mean Pooling

Architecture

- 3-layer GIN with MLP-based aggregation
- Hidden dimension: 64
- Batch Normalization + ReLU after each layer

Similar graphs → similar embeddings in vector space

Graph Embedding using GIN

Graph

We use a Graph Isomorphism Network (GIN) to learn graph-level embeddings that capture structural similarity between graphs.

Graph Representation

- Node features updated using neighborhood aggregation
- Graph embedding obtained via Global Mean Pooling

Architecture

- 3-layer GIN with MLP-based aggregation
- Hidden dimension: 64
- Batch Normalization + ReLU after each layer

Similar graphs → similar embeddings in vector space

Graph Embedding using GIN

We use a Graph Isomorphism Network (GIN) to learn graph-level embeddings that capture structural similarity between graphs.

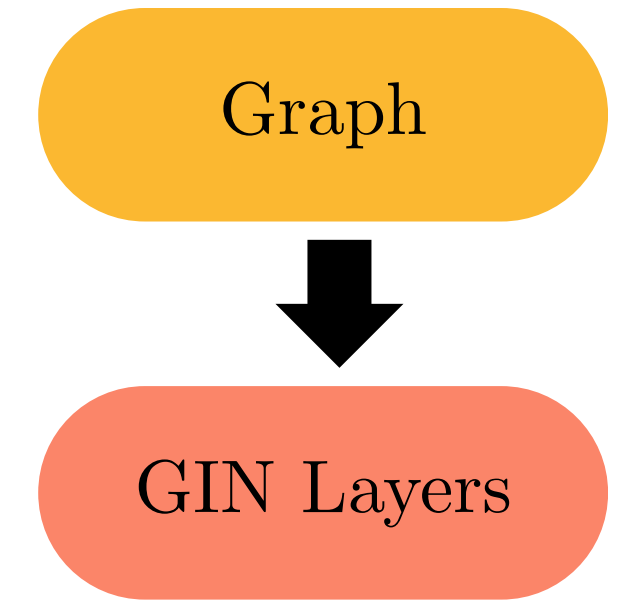
Graph Representation

- Node features updated using neighborhood aggregation
- Graph embedding obtained via Global Mean Pooling

Architecture

- 3-layer GIN with MLP-based aggregation
- Hidden dimension: 64
- Batch Normalization + ReLU after each layer

Similar graphs → similar embeddings in vector space



Graph Embedding using GIN

We use a Graph Isomorphism Network (GIN) to learn graph-level embeddings that capture structural similarity between graphs.

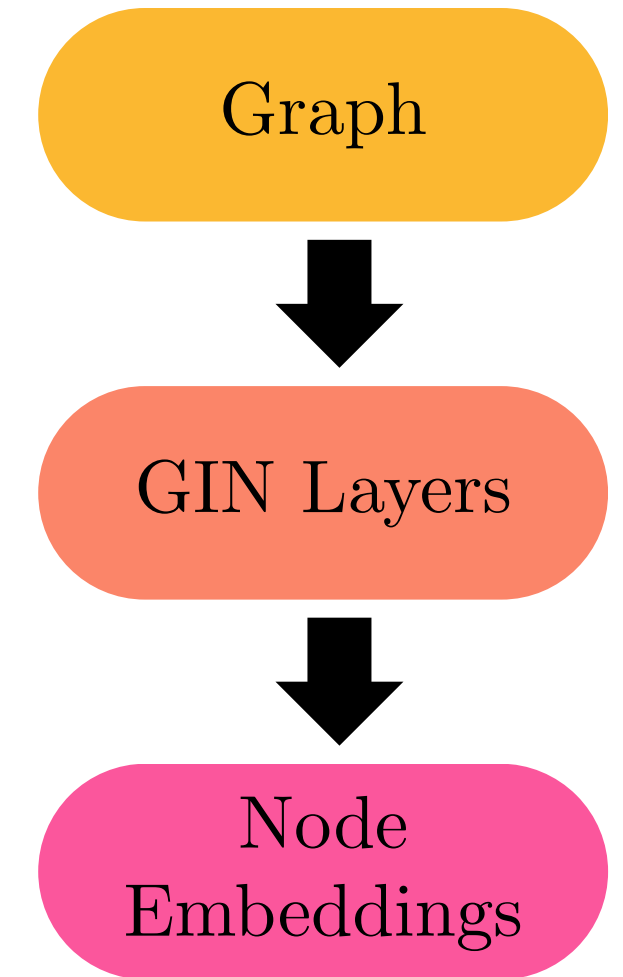
Graph Representation

- Node features updated using neighborhood aggregation
- Graph embedding obtained via Global Mean Pooling

Architecture

- 3-layer GIN with MLP-based aggregation
- Hidden dimension: 64
- Batch Normalization + ReLU after each layer

Similar graphs → similar embeddings in vector space



Graph Embedding using GIN

We use a Graph Isomorphism Network (GIN) to learn graph-level embeddings that capture structural similarity between graphs.

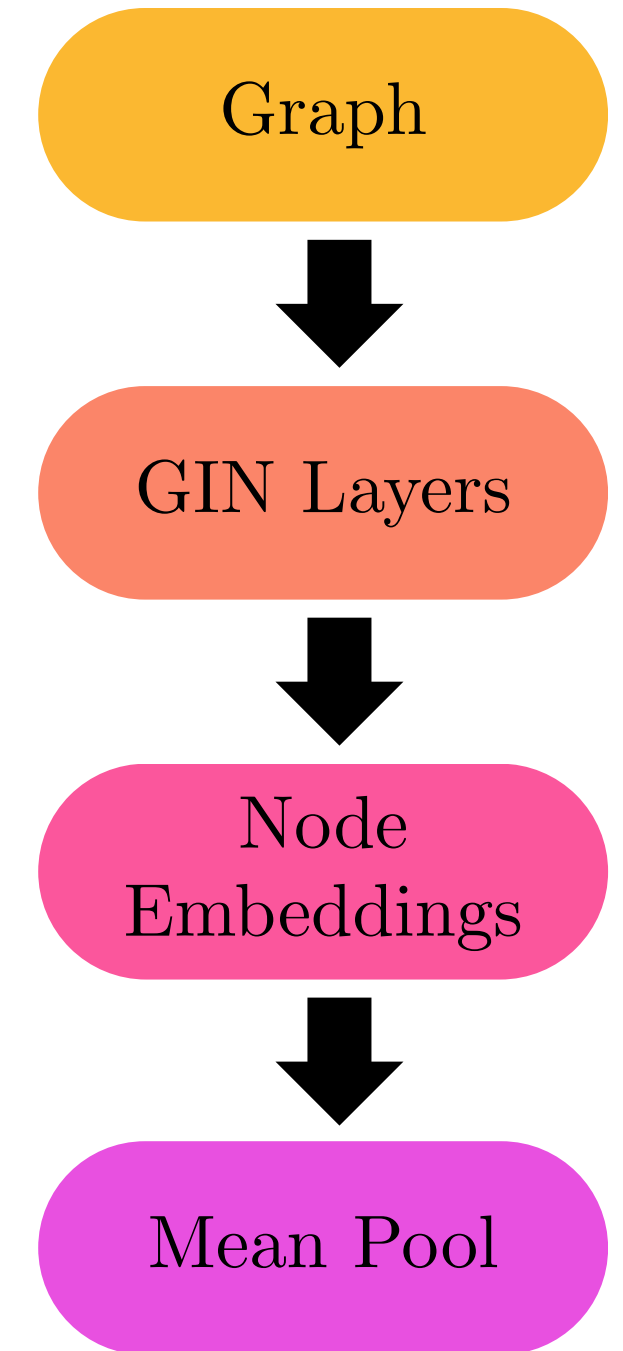
Graph Representation

- Node features updated using neighborhood aggregation
- Graph embedding obtained via Global Mean Pooling

Architecture

- 3-layer GIN with MLP-based aggregation
- Hidden dimension: 64
- Batch Normalization + ReLU after each layer

Similar graphs → similar embeddings in vector space



Graph Embedding using GIN

We use a Graph Isomorphism Network (GIN) to learn graph-level embeddings that capture structural similarity between graphs.

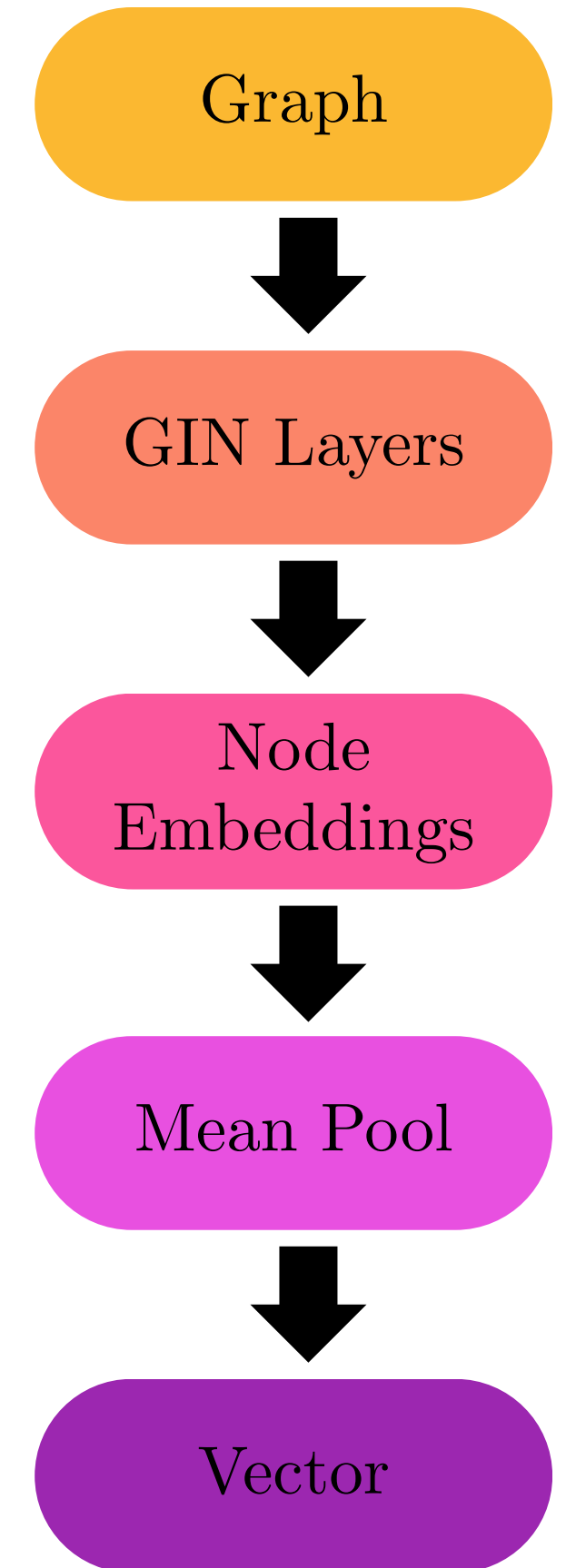
Graph Representation

- Node features updated using neighborhood aggregation
- Graph embedding obtained via Global Mean Pooling

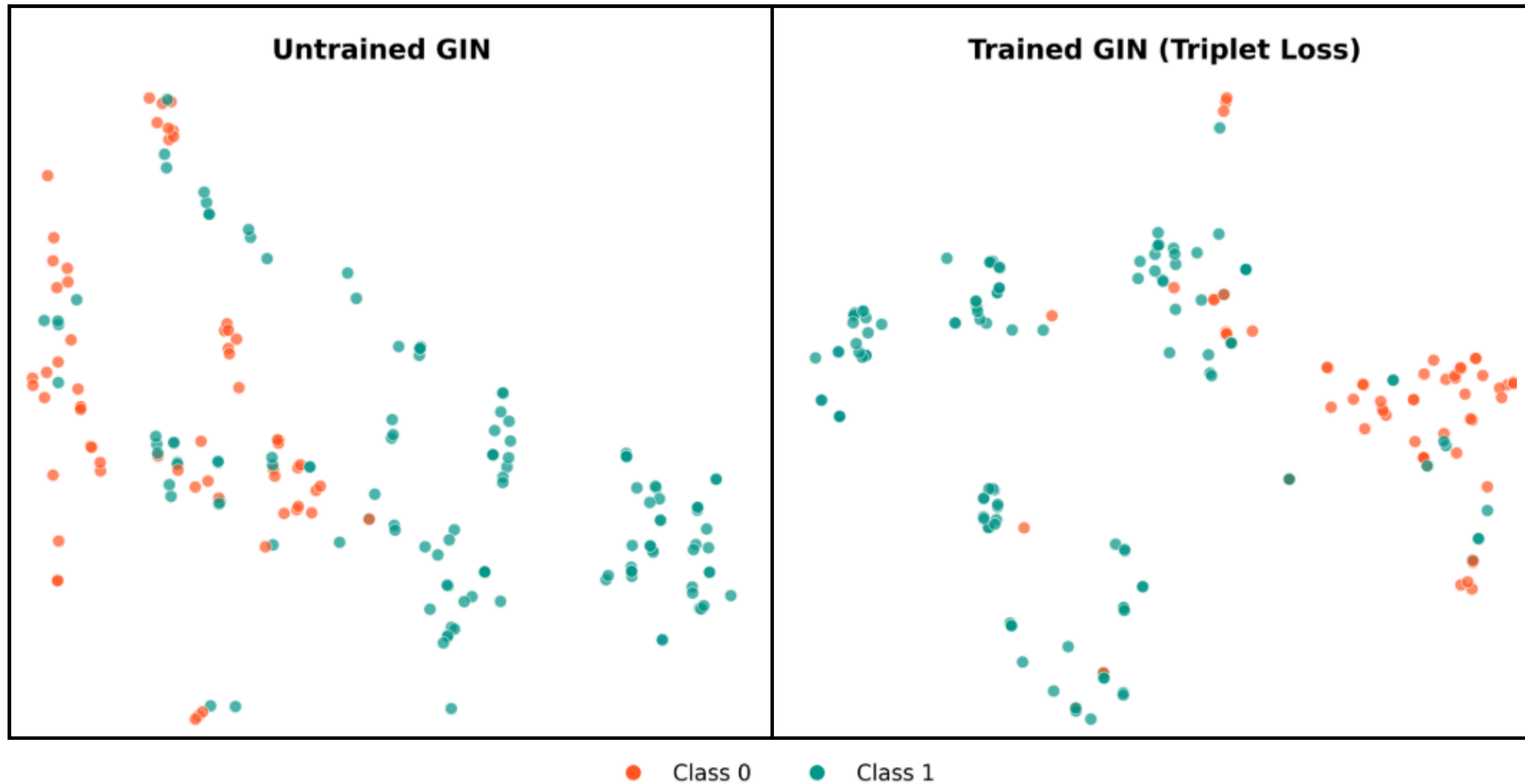
Architecture

- 3-layer GIN with MLP-based aggregation
- Hidden dimension: 64
- Batch Normalization + ReLU after each layer

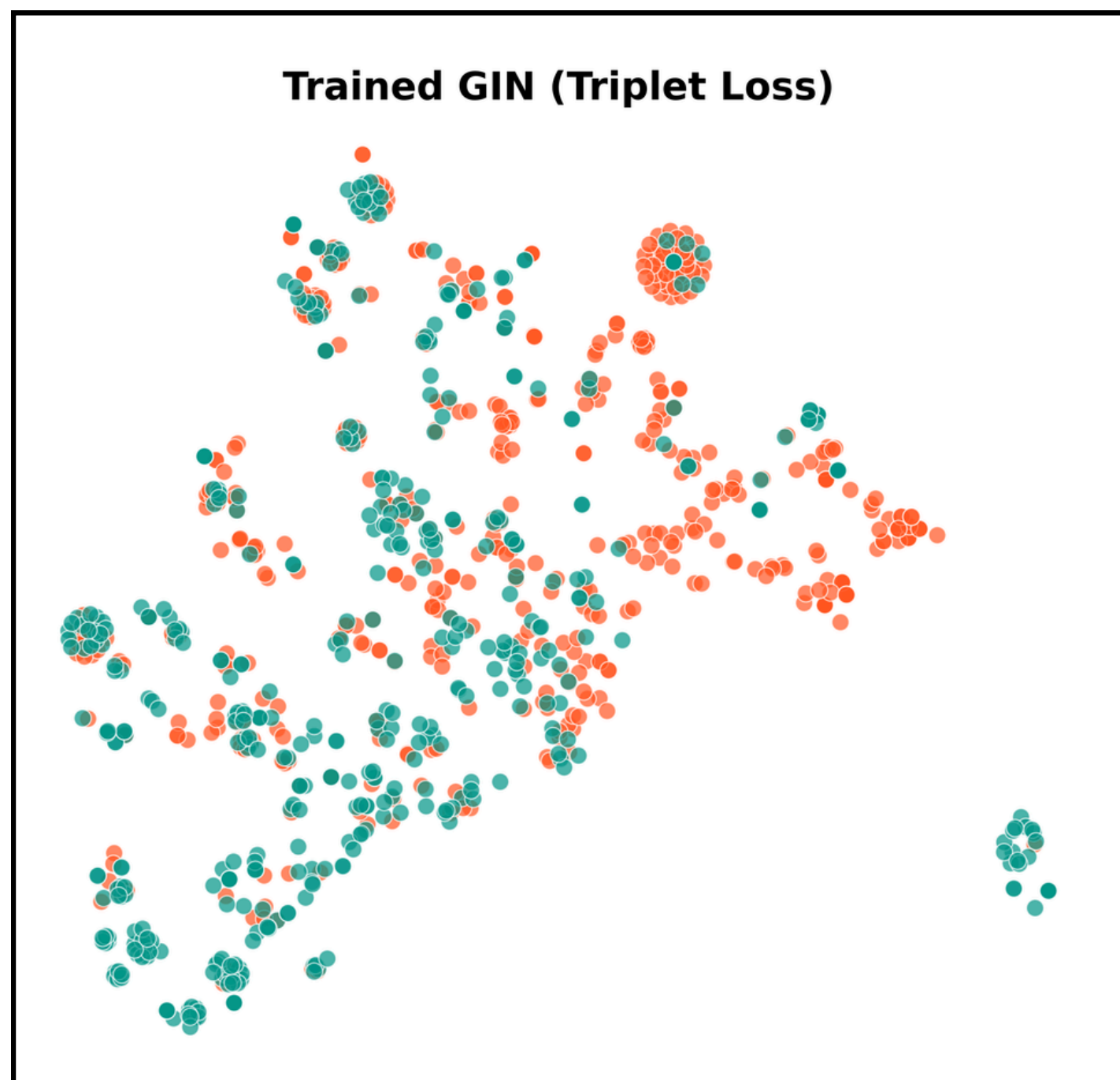
Similar graphs → similar embeddings in vector space



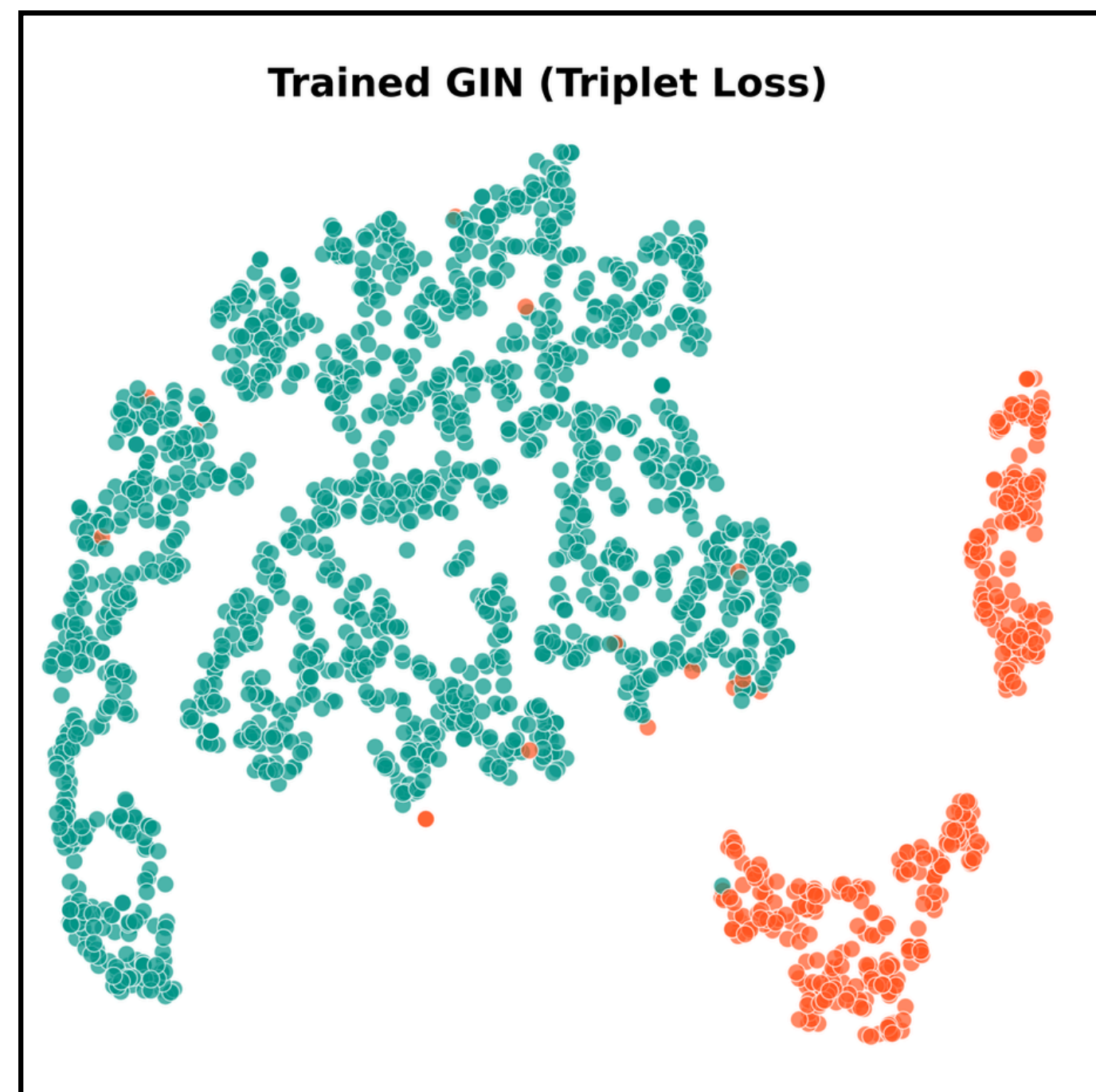
t-SNE Visualization of MUTAG Embeddings



t-SNE Visualization

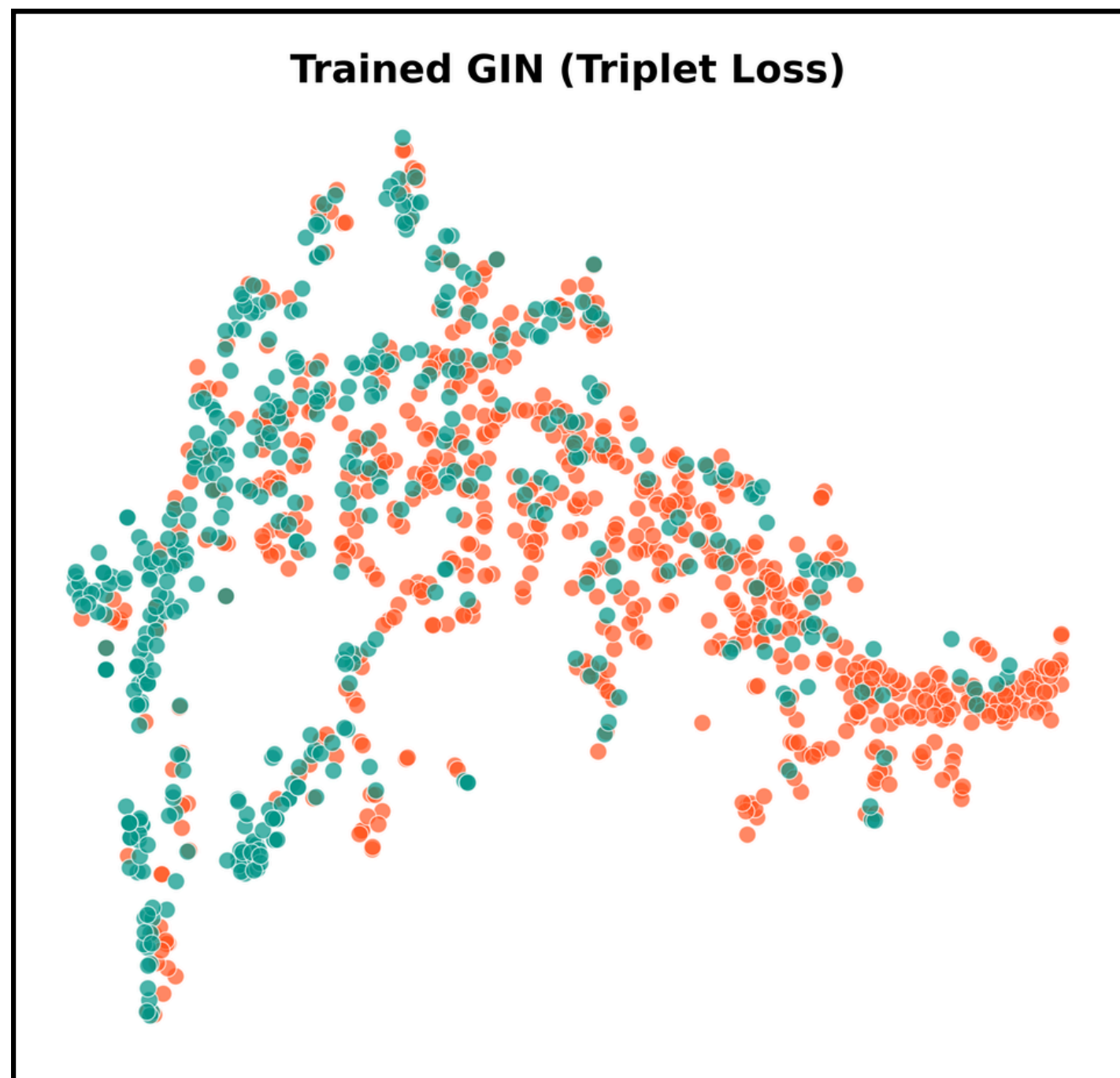


IMDB-binary Dataset

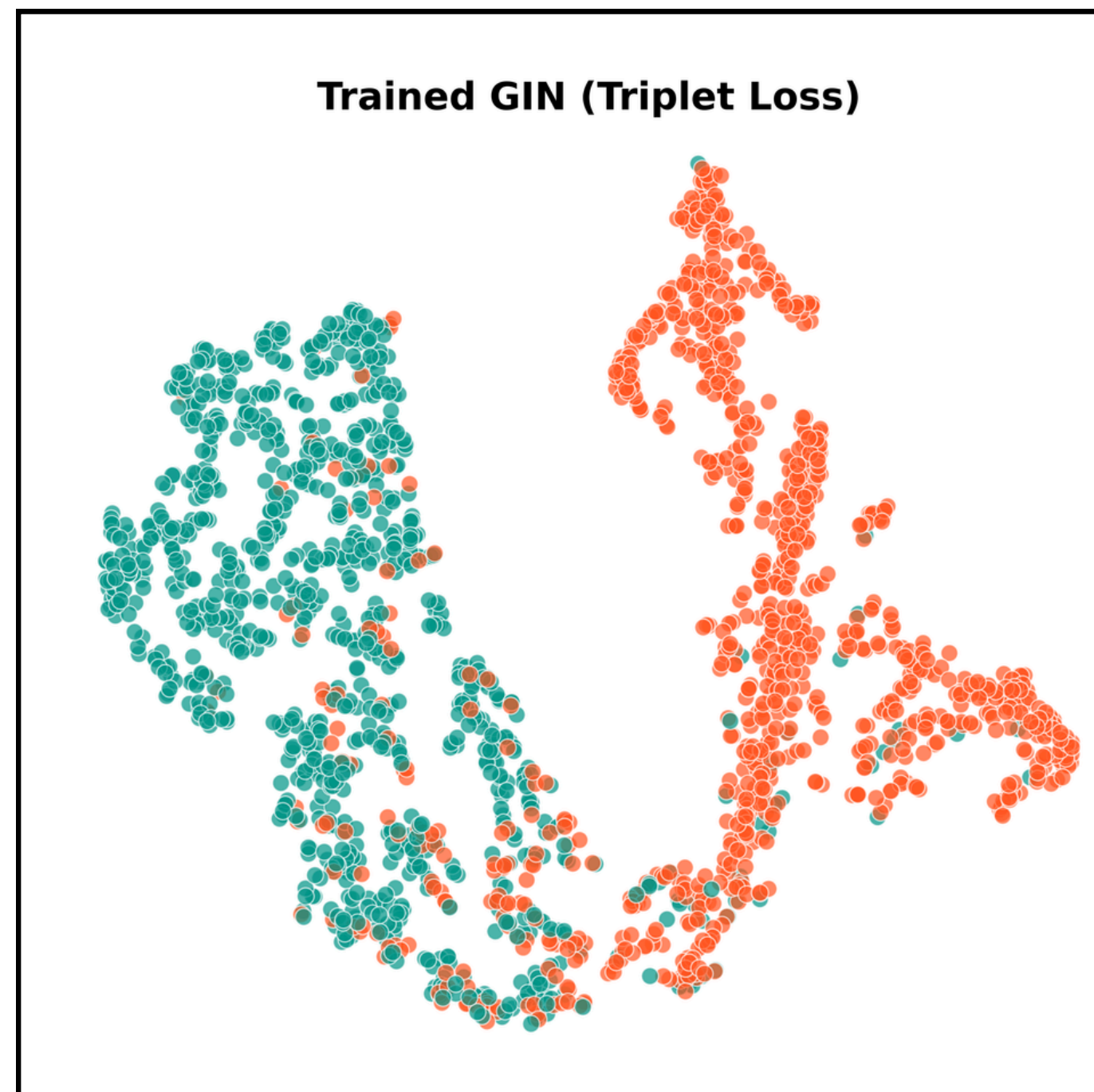


Aids Dataset

t-SNE Visualization



Protein Dataset



Reddit-Binary Dataset

Training with Triplet Loss

$$L = \max(0, \|z_Q - z_P\|_2 - \|z_Q - z_N\|_2 + \alpha)$$

Idea

Similar graphs are closer

Dissimilar graphs are farther apart

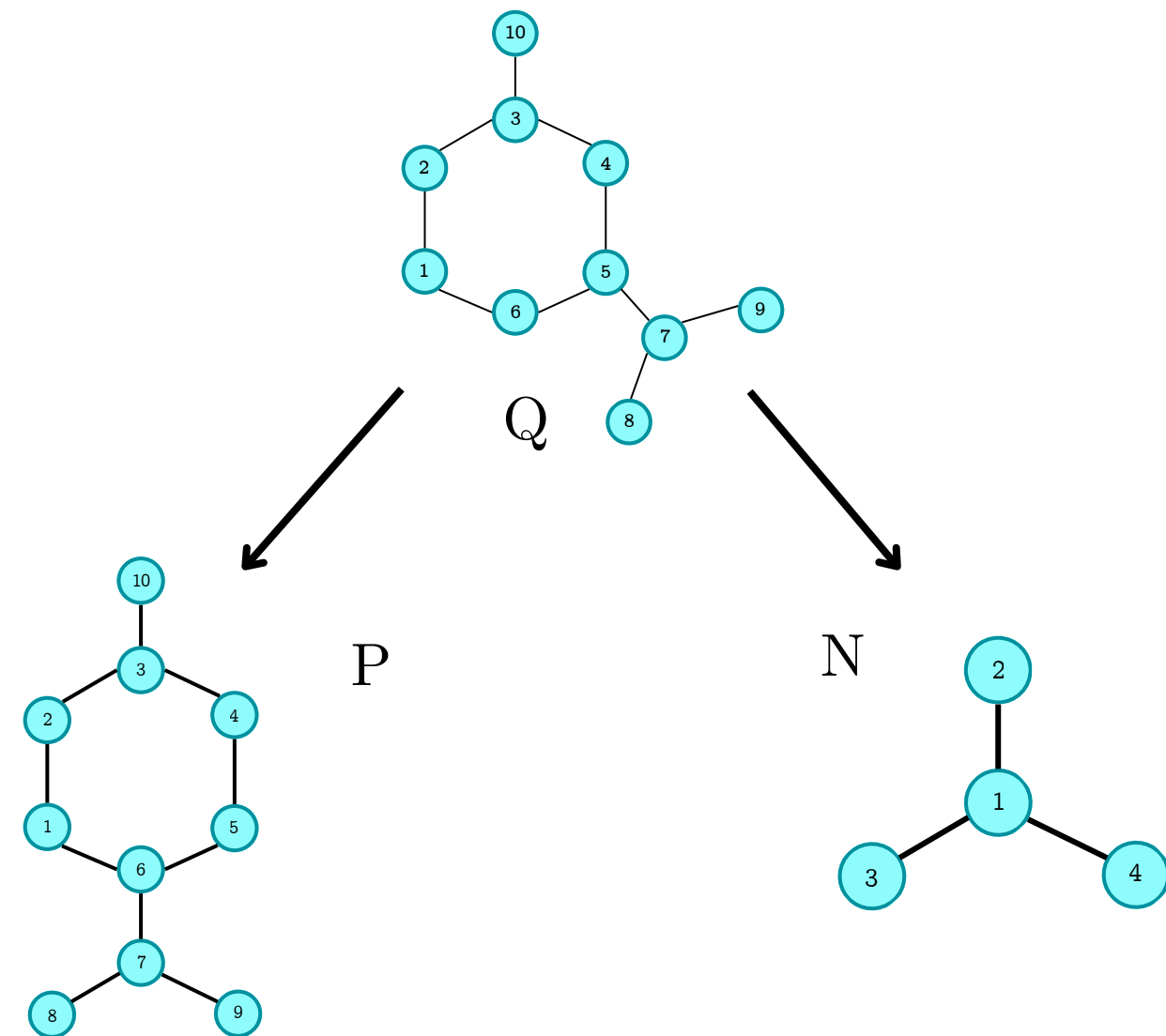
Triplet Setup

Each training sample consists of:

- Anchor (Q): Query graph
- Positive (P): Similar graph
- Negative (N): Dissimilar graph

Ensure:

$$\|z_Q - z_P\| < \|z_Q - z_N\|$$



GED Oracle Separation

Training oracle

Beam search GED (B=5) — fast approximation, used to generate triplet labels during training.

Evaluation oracle

Exact GED for MUTAG, Beam search GED (B=20) for larger datasets — higher quality ground truth.

Why separate? Using the same oracle for both training and evaluation inflates results artificially. The model would simply learn to agree with its own training labels rather than capturing true structural similarity.

This ensures reported Precision@k and MAP numbers reflect genuine retrieval quality, not self-consistency.

Locality Sensitive Hashing

Hash Function

$$h(z) = \left\lfloor \frac{a \cdot z + b}{w} \right\rfloor$$

- $a \sim N(0, I)$: random projection
- $b \sim U(0, w)$: random shift
- w : bucket width

MultiHash Strategy

- K projections / table $\rightarrow$ sharper buckets ($\uparrow$ precision)
- L hash tables $\rightarrow$ more chances to retrieve neighbors ($\uparrow$ recall)

Search is performed only within the candidate set, not the entire dataset.

Locality Sensitive Hashing

Query
Embedding

Hash Function

$$h(z) = \left\lfloor \frac{a \cdot z + b}{w} \right\rfloor$$

- $a \sim N(0, I)$: random projection
- $b \sim U(0, w)$: random shift
- w : bucket width

MultiHash Strategy

- K projections / table $\rightarrow$ sharper buckets ($\uparrow$ precision)
- L hash tables $\rightarrow$ more chances to retrieve neighbors ($\uparrow$ recall)

Search is performed only within the candidate set, not the entire dataset.

Locality Sensitive Hashing

Hash Function

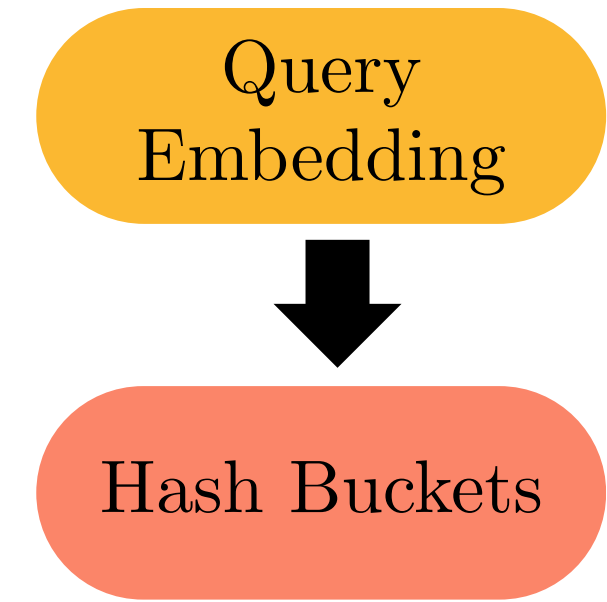
$$h(z) = \left\lceil \frac{a \cdot z + b}{w} \right\rceil$$

- $a \sim N(0, I)$: random projection
- $b \sim U(0, w)$: random shift
- w : bucket width

MultiHash Strategy

- K projections / table $\rightarrow$ sharper buckets ($\uparrow$ precision)
- L hash tables $\rightarrow$ more chances to retrieve neighbors ($\uparrow$ recall)

Search is performed only within the candidate set, not the entire dataset.



Locality Sensitive Hashing

Hash Function

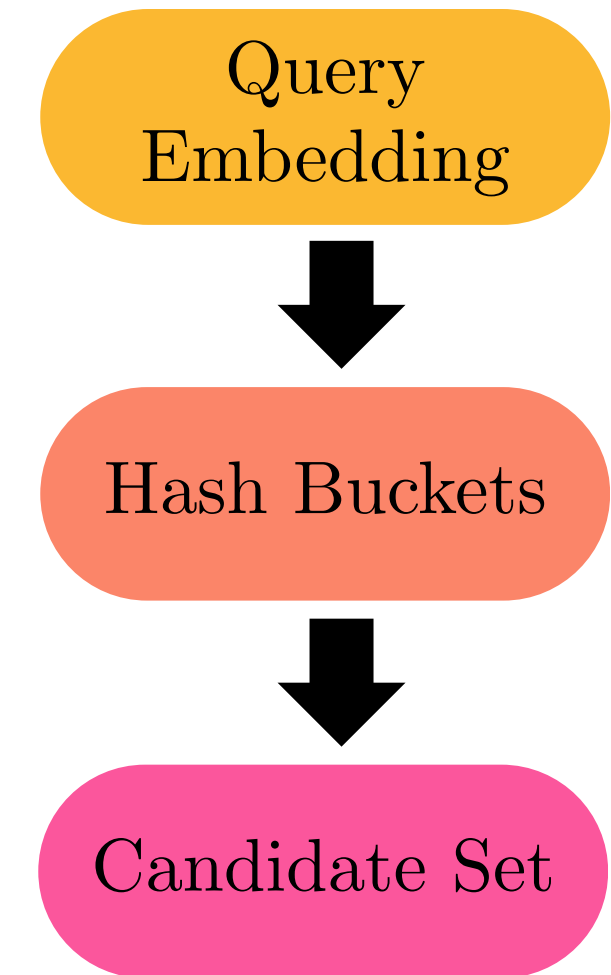
$$h(z) = \left\lceil \frac{a \cdot z + b}{w} \right\rceil$$

- $a \sim N(0, I)$: random projection
- $b \sim U(0, w)$: random shift
- w : bucket width

MultiHash Strategy

- K projections / table $\rightarrow$ sharper buckets ($\uparrow$ precision)
- L hash tables $\rightarrow$ more chances to retrieve neighbors ($\uparrow$ recall)

Search is performed only within the candidate set, not the entire dataset.



Locality Sensitive Hashing

Hash Function

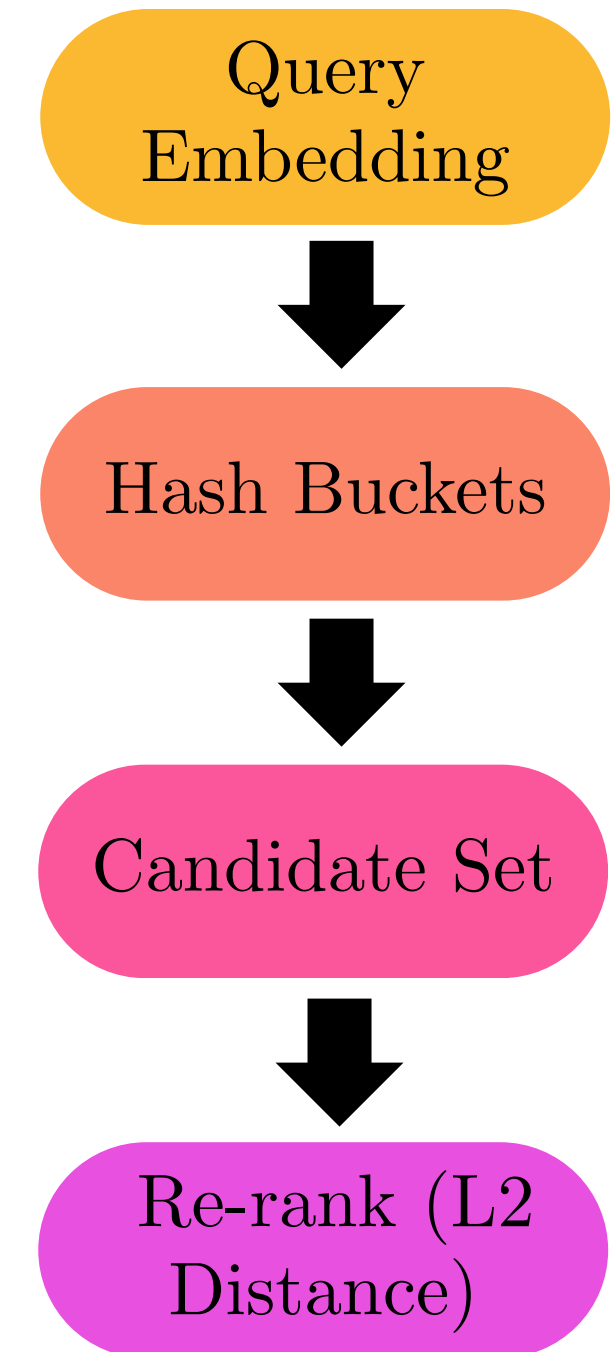
$$h(z) = \left\lfloor \frac{a \cdot z + b}{w} \right\rfloor$$

- $a \sim N(0, I)$: random projection
- $b \sim U(0, w)$: random shift
- w : bucket width

MultiHash Strategy

- K projections / table $\rightarrow$ sharper buckets ($\uparrow$ precision)
- L hash tables $\rightarrow$ more chances to retrieve neighbors ($\uparrow$ recall)

Search is performed only within the candidate set, not the entire dataset.



Locality Sensitive Hashing

Hash Function

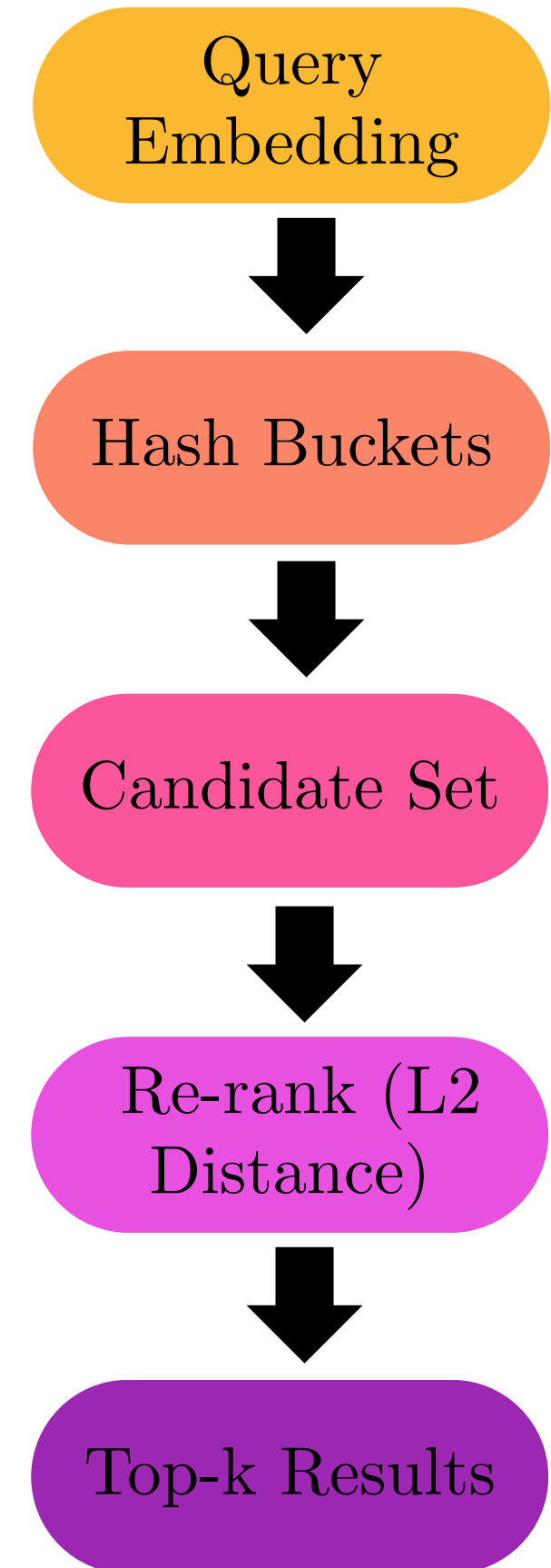
$$h(z) = \left\lceil \frac{a \cdot z + b}{w} \right\rceil$$

- $a \sim N(0, I)$: random projection
- $b \sim U(0, w)$: random shift
- w : bucket width

MultiHash Strategy

- K projections / table $\rightarrow$ sharper buckets ($\uparrow$ precision)
- L hash tables $\rightarrow$ more chances to retrieve neighbors ($\uparrow$ recall)

Search is performed only within the candidate set, not the entire dataset.



Baselines for Comparison

Exact exhaustive search using the trained GIN embeddings to measure the cost of LSH approximation.

Brute-Force

Untrained-ANN

LSH indexing over randomly initialized GIN embeddings to isolate the value of contrastive training.

An unsupervised structural baseline utilizing Weisfeiler-Leman subtree features.

Graph2vec-ANN

Evaluation Metrics

- **Precision@k**

→ relevance of retrieved results

$$\text{Precision@k} = \frac{|R_k \cap R^*|}{k}$$

- R_k : top-k retrieved items
- R^* : set of relevant items

- **Recall@k**

→ coverage of relevant results

$$\text{Precision@k} = \frac{|R_k \cap R^*|}{|R^*|}$$

- R_k : top-k retrieved items
- R^* : set of relevant items

- **Average Precision (AP)**

→ ranking quality

$$AP = \frac{1}{|R^*|} \sum_{i=1}^k P(i) \cdot 1_{\{\text{item at } i \text{ relevant}\}} \quad P(i) : \text{Precision@}i$$

Evaluation Metrics

- **Mean Average Precision (MAP)** → ranking quality

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

Q : set of queries

- **Approximation Quality (AQ)** → LSH vs exact

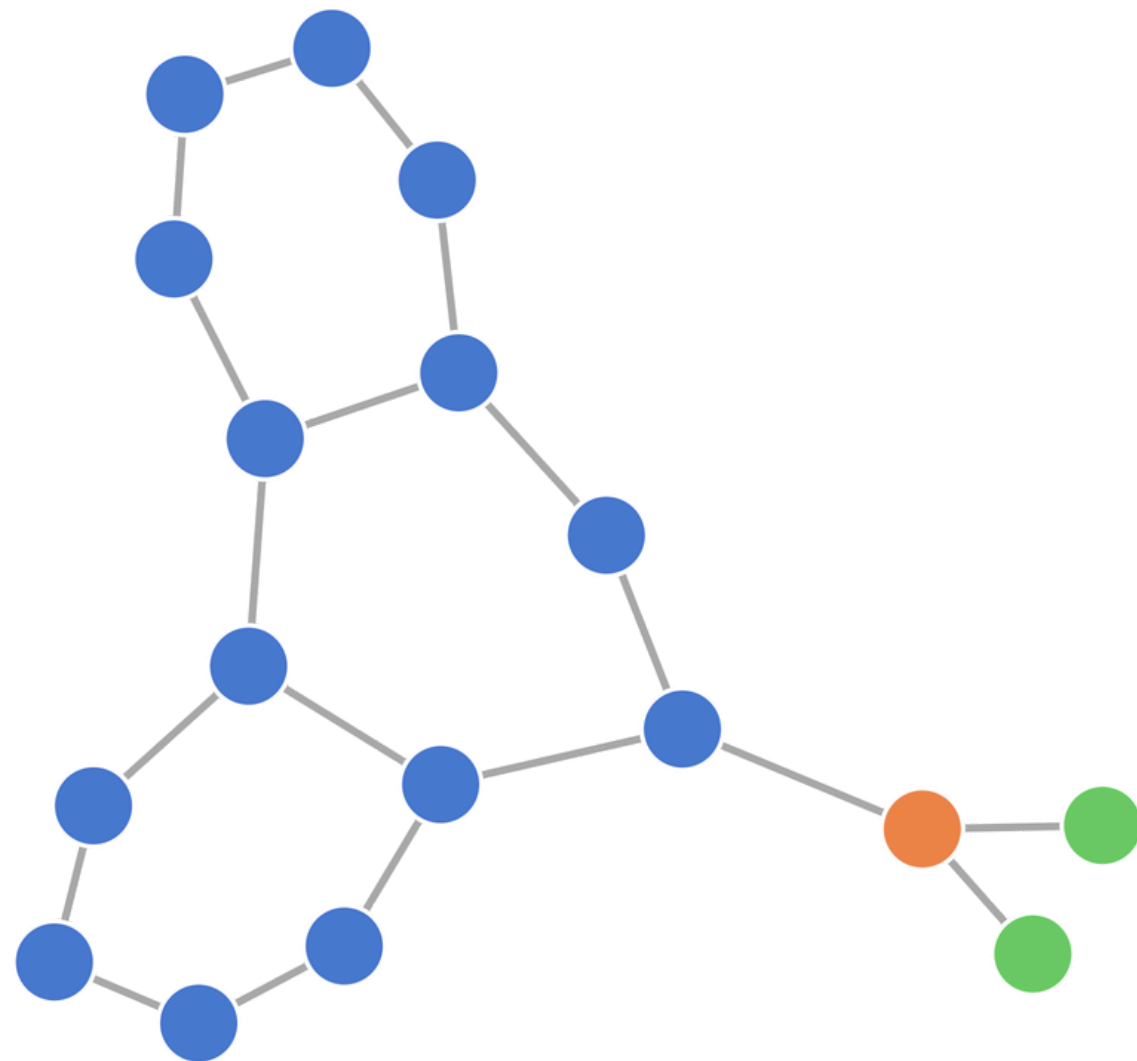
$$\text{Precision@k} = \frac{|R_k^{\text{LSH}} \cap R_k^{\text{BF}}|}{k}$$

- R_k : top-k retrieved items
- R^* : set of relevant items

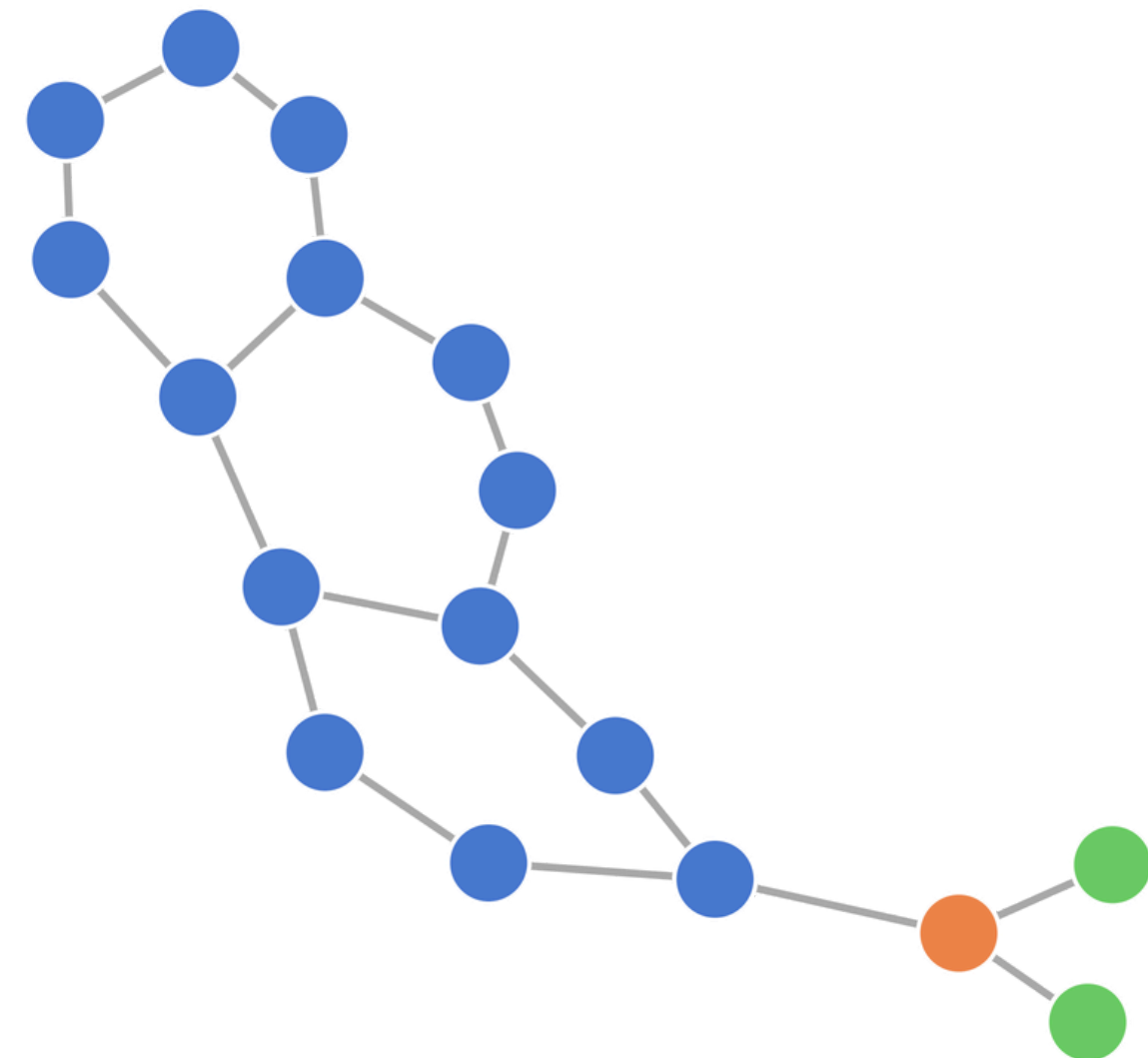
- **Query Time** → efficiency

Query Time=Time to retrieve top-k results

Example Retrieval MUTAG

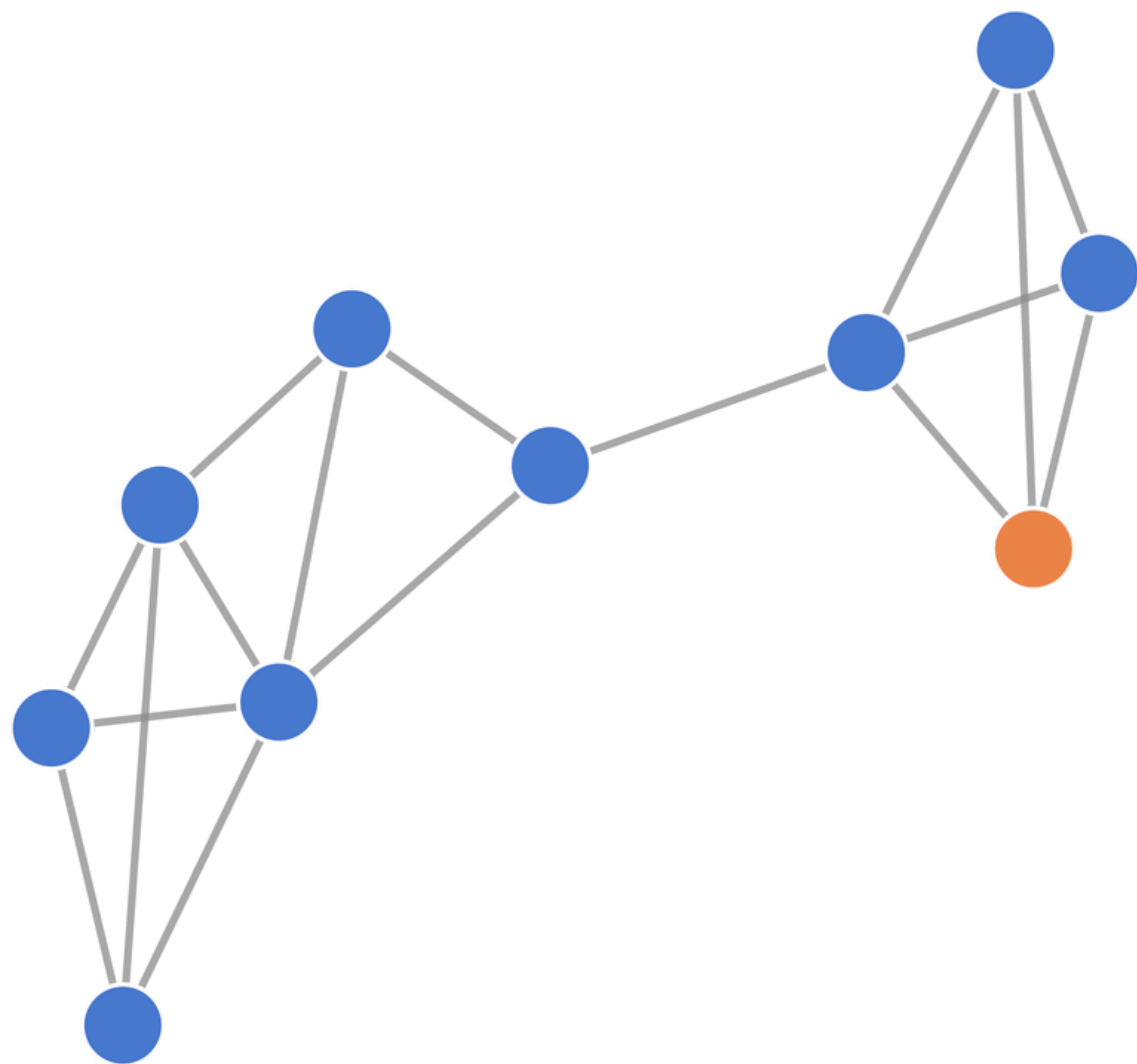


Query

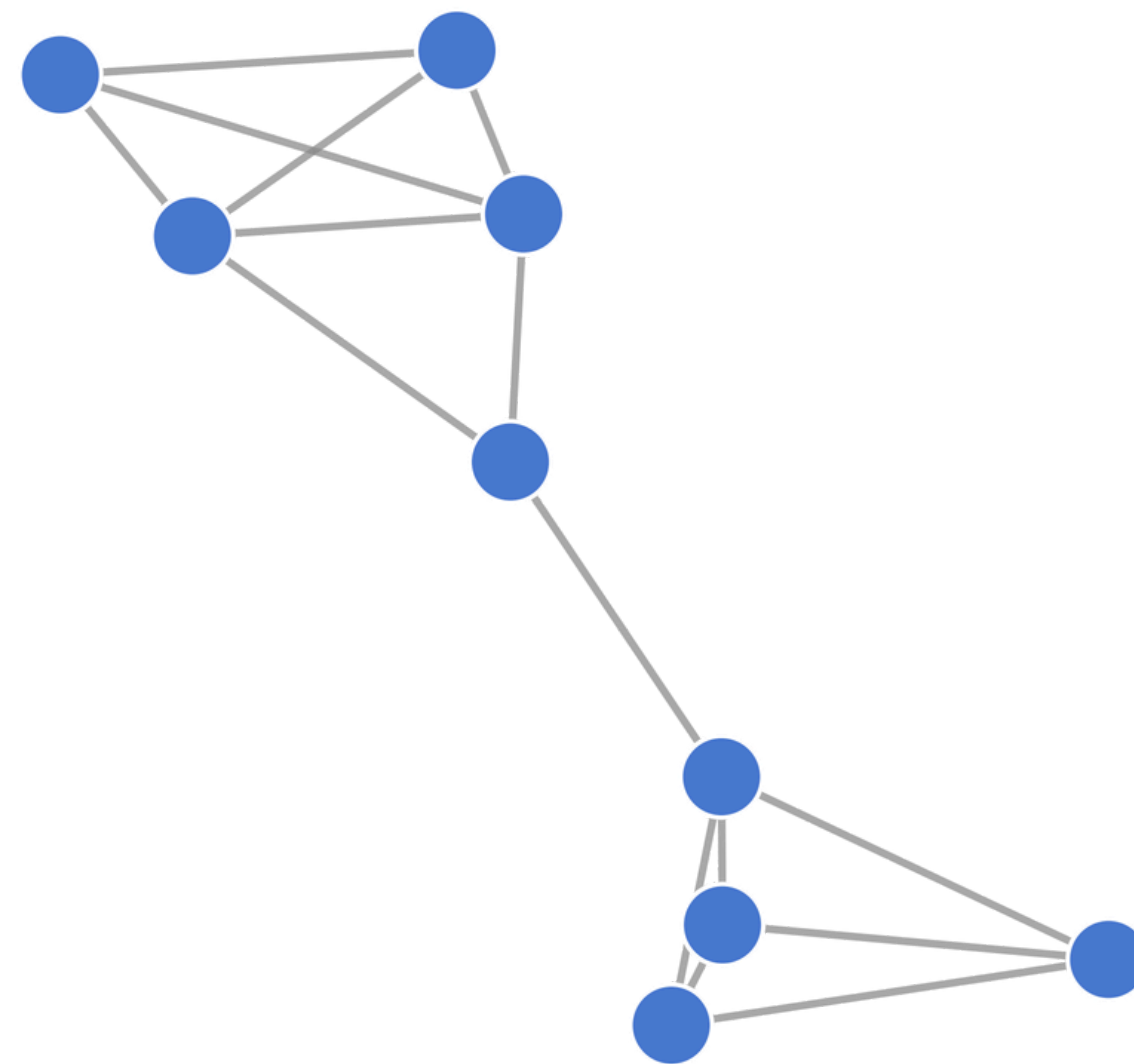


Retrieved Graph

Example Retrieval Proteins

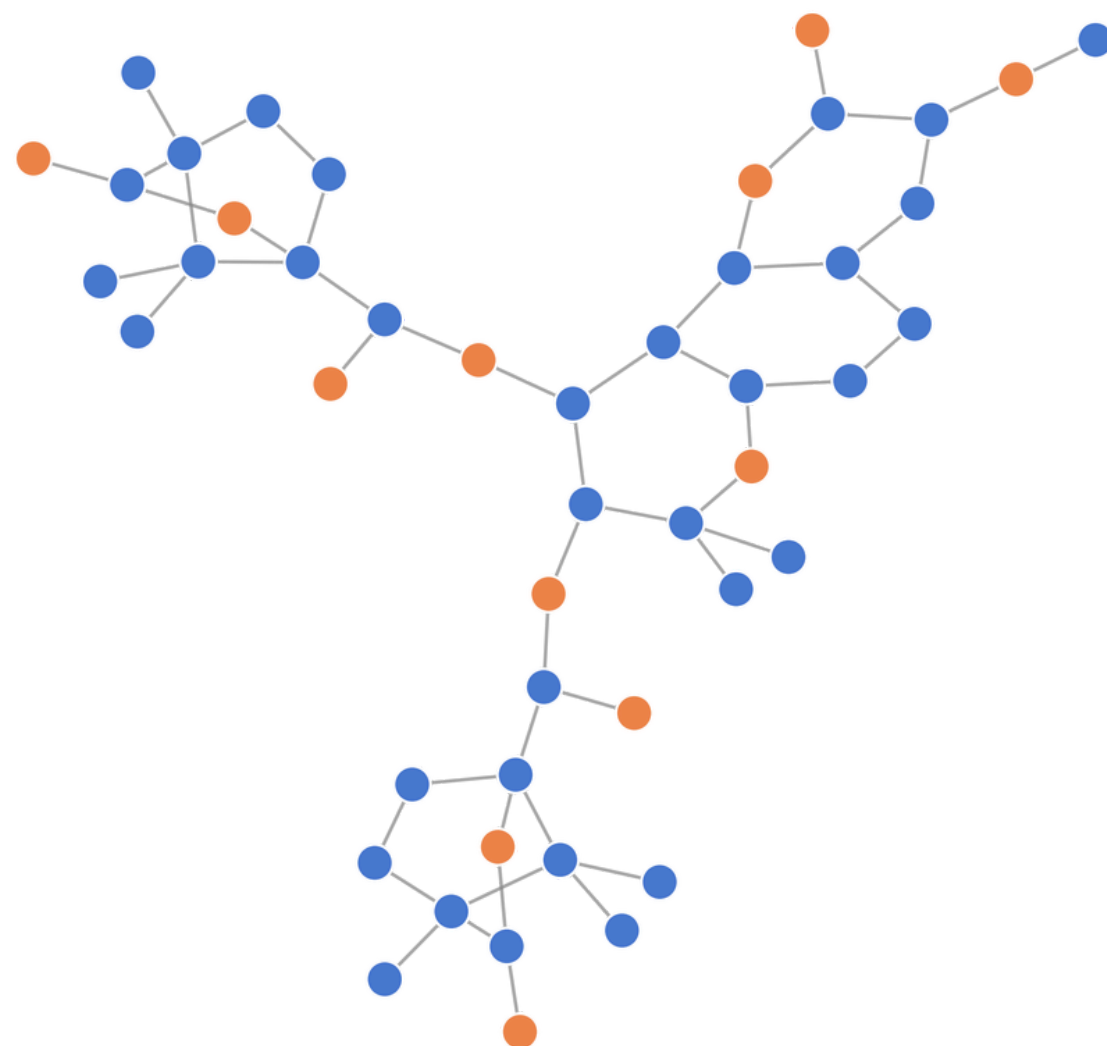


Query

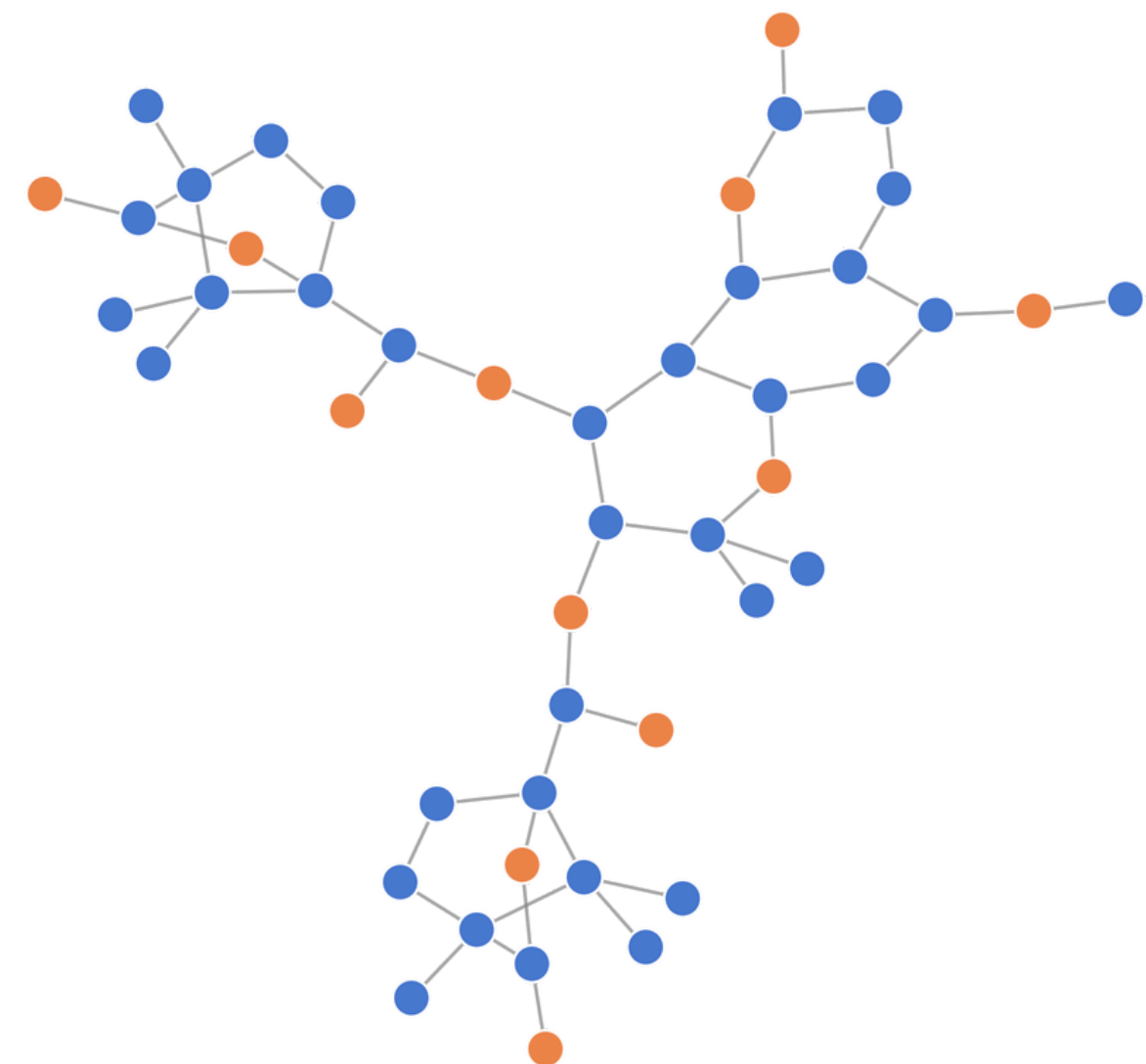


Retrieved Graph

Example Retrieval AIDS

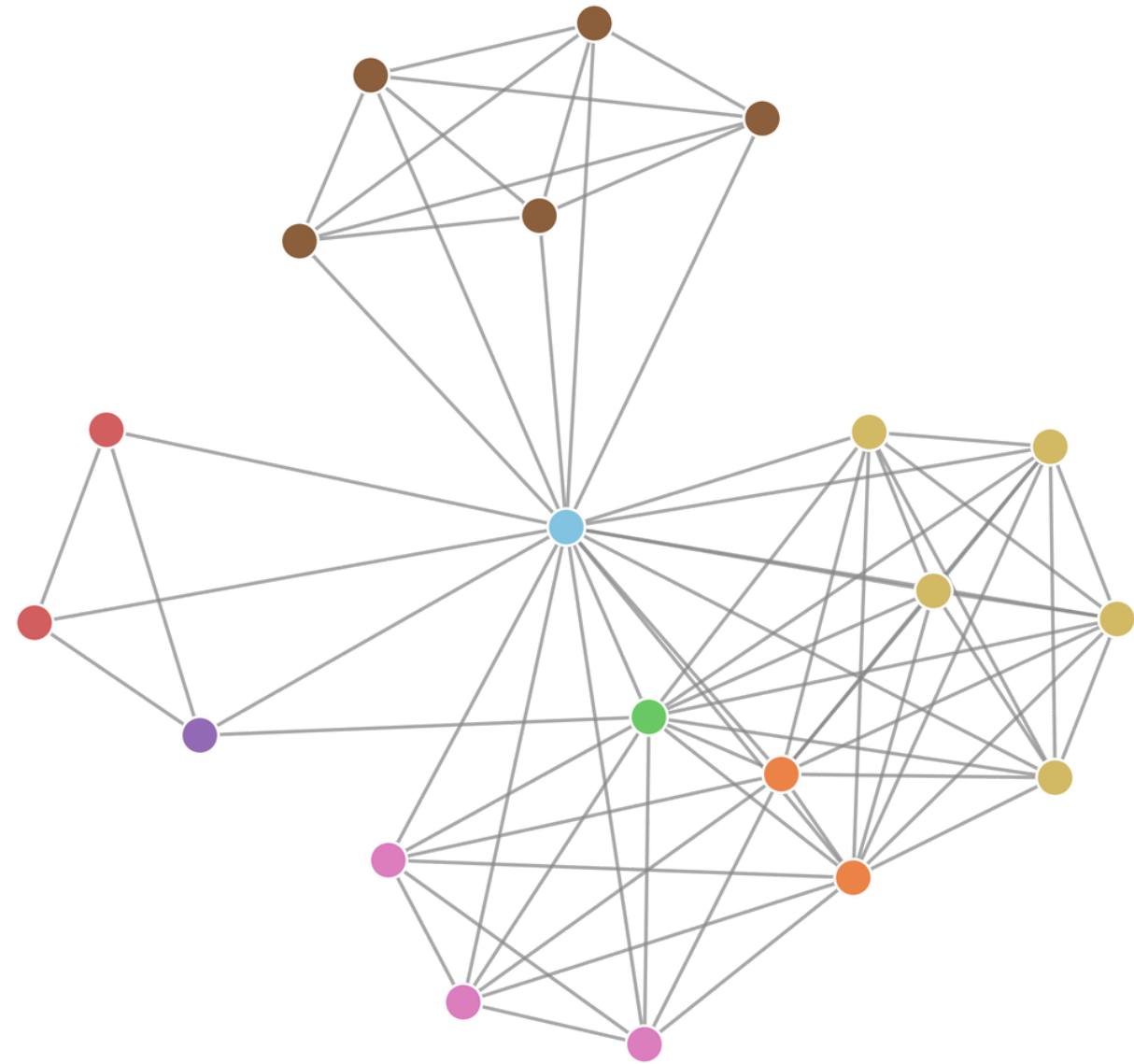


Query

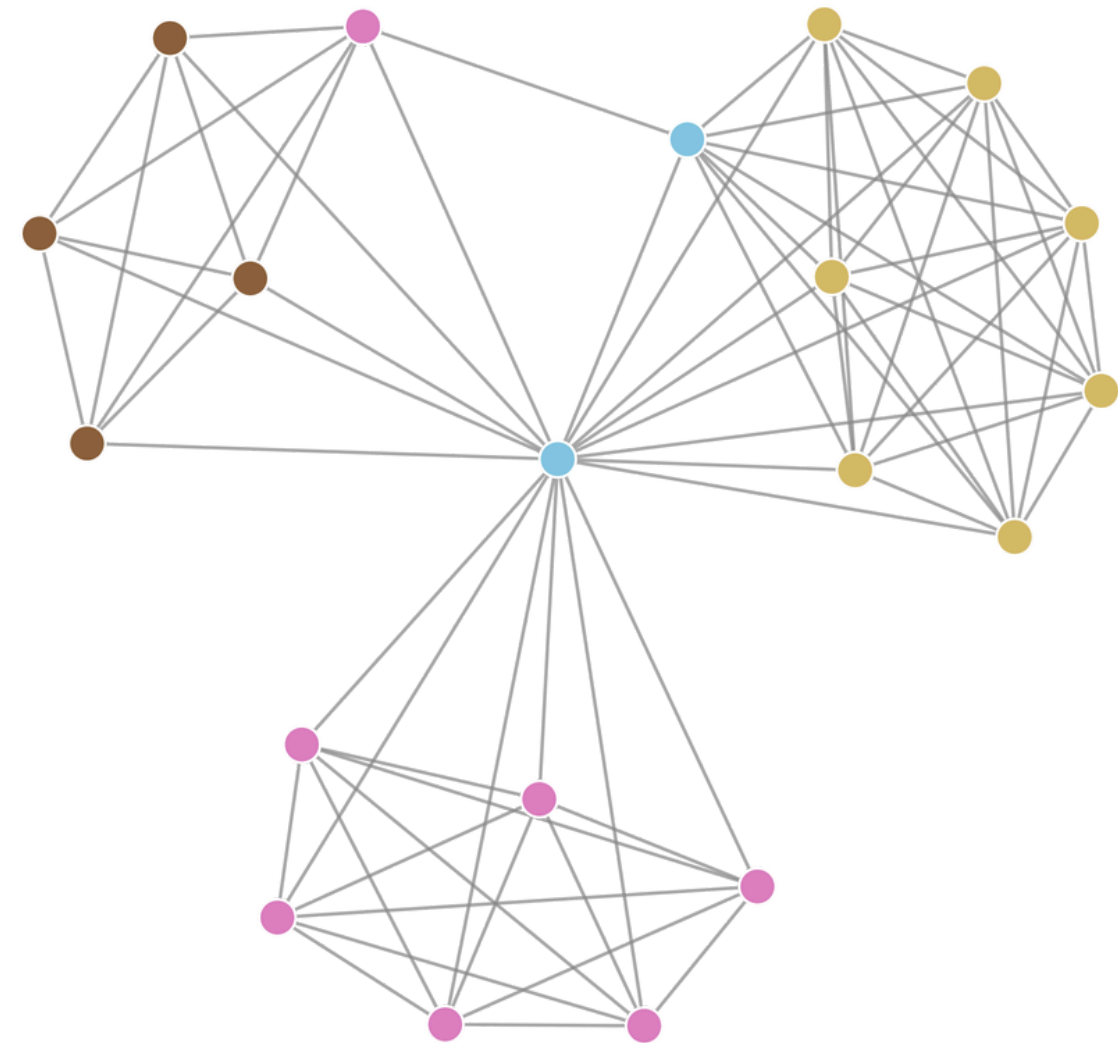


Retrieved Graph

Example Retrieval IMDB-Binary

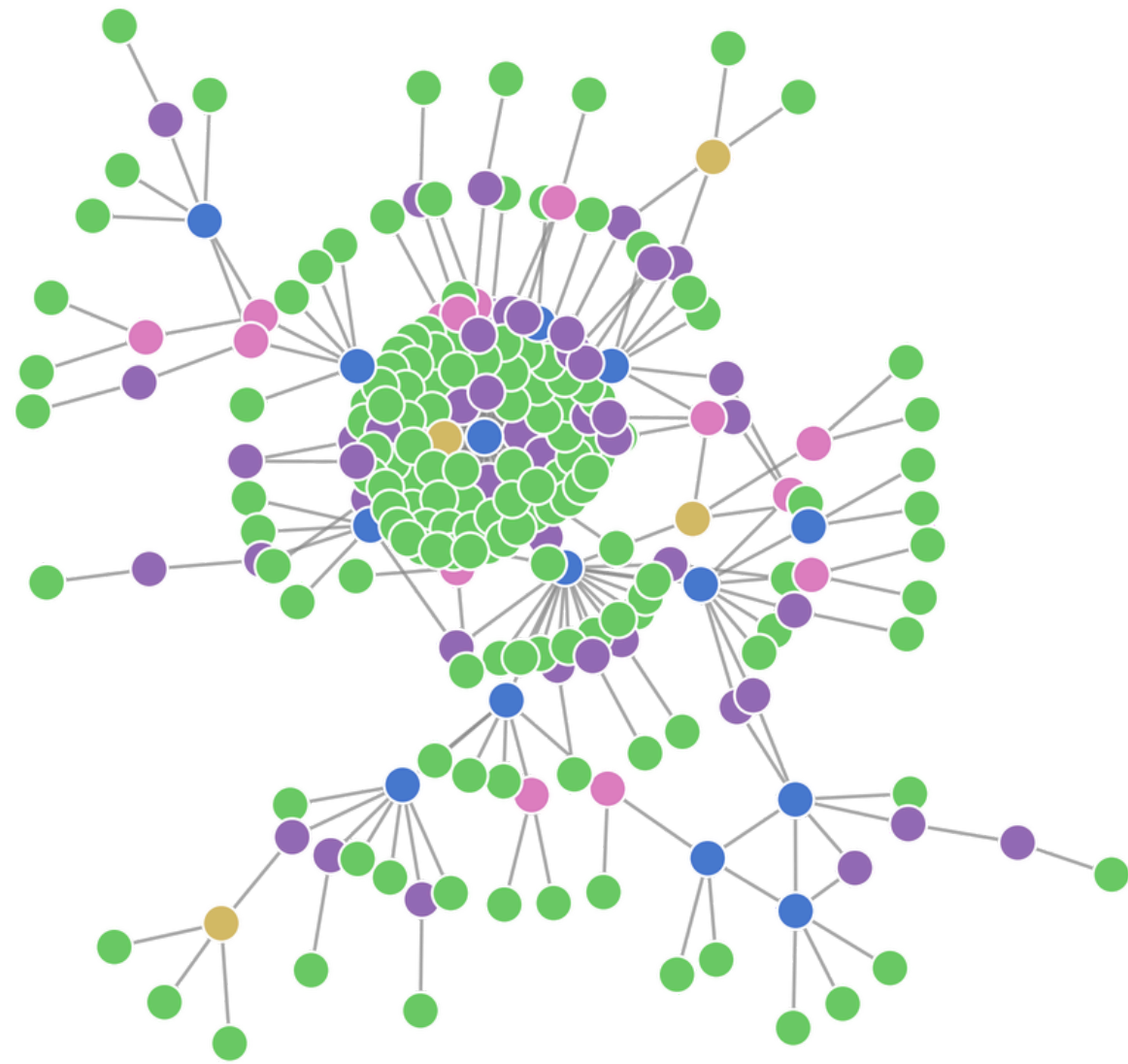


Query

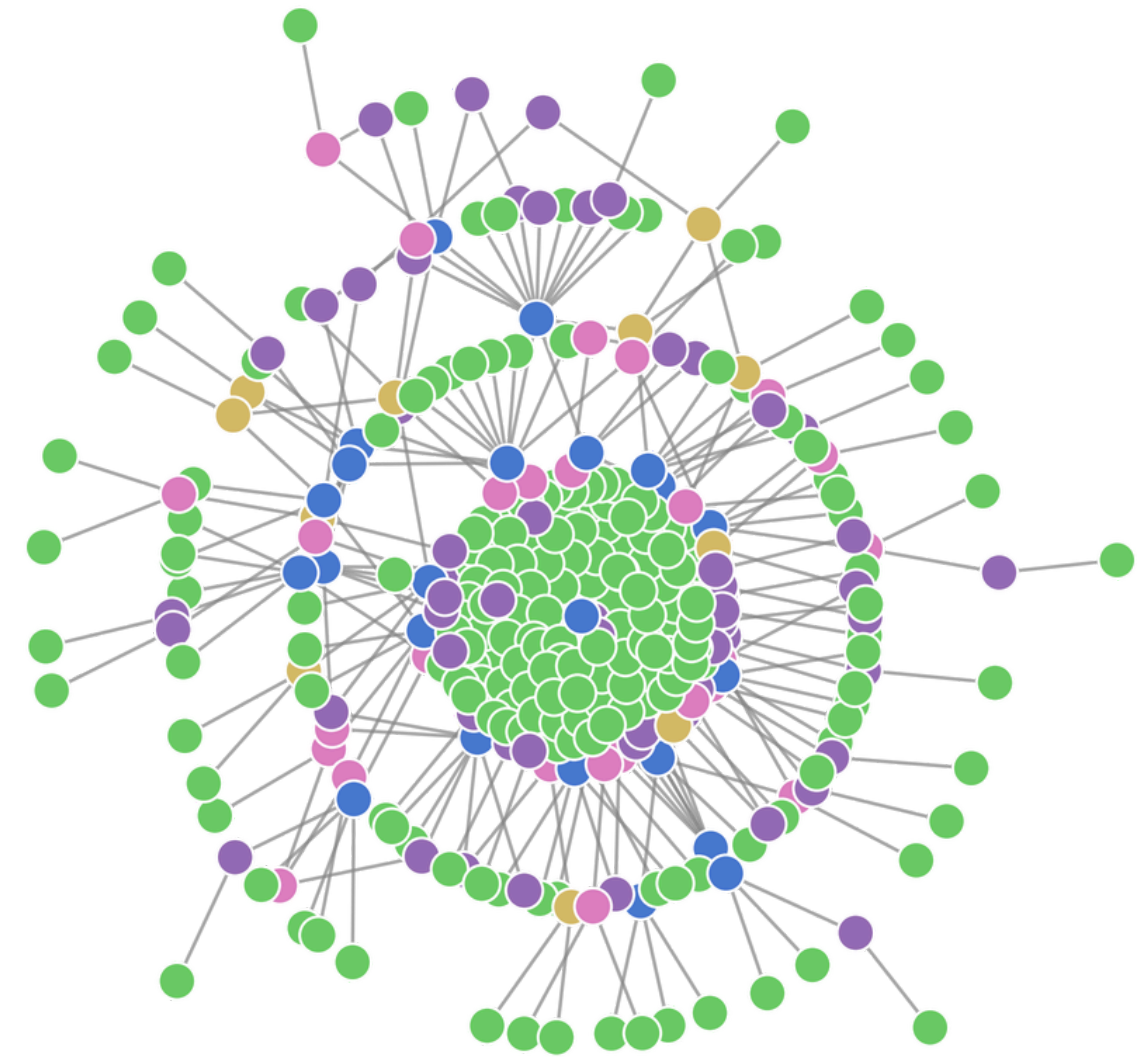


Retrieved Graph

Example Retrieval Reddit-Binary



Query

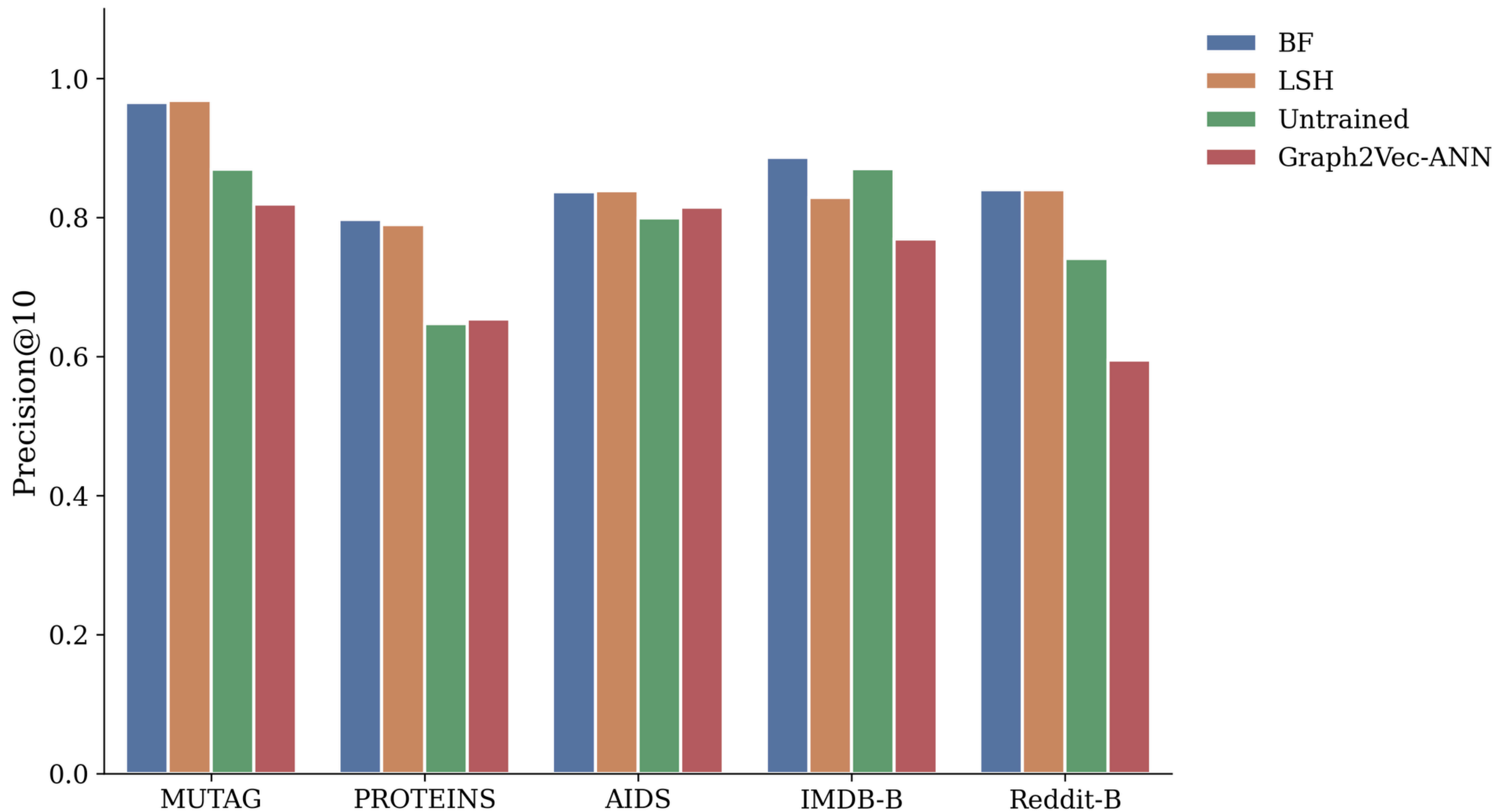


Retrieved Graph

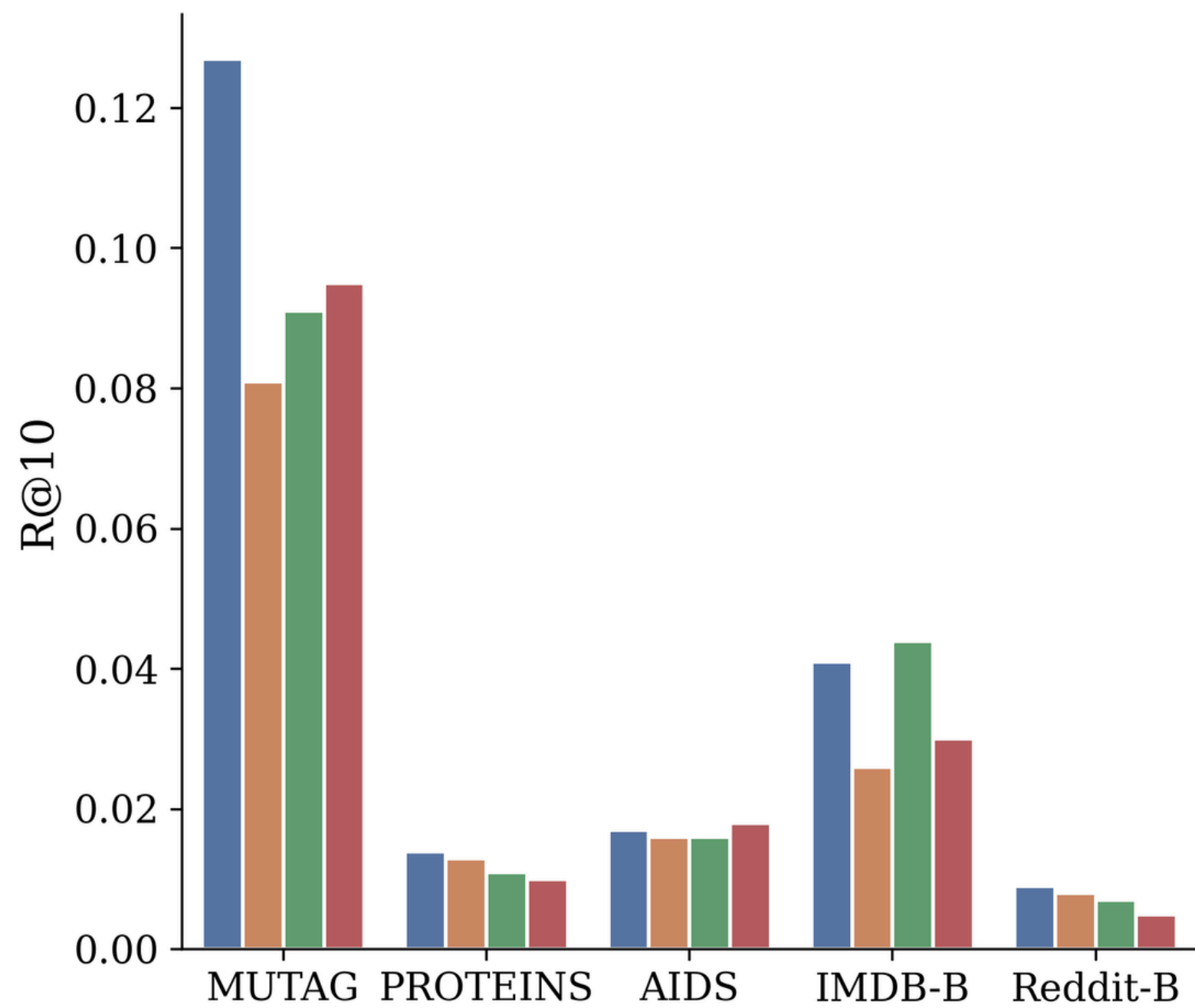
Results : Precision@10

Datasets	Brute Force	Untrained ANN	Graph2Vec	Trained LSH
MUTAG	0.965	0.869	0.819	0.968
PROTEIN	0.797	0.647	0.654	0.790
AIDS	0.837	0.799	0.815	0.838
IMDB-Binary	0.886	0.870	0.769	0.829
Reddit-Binary	0.840	0.741	0.595	0.840

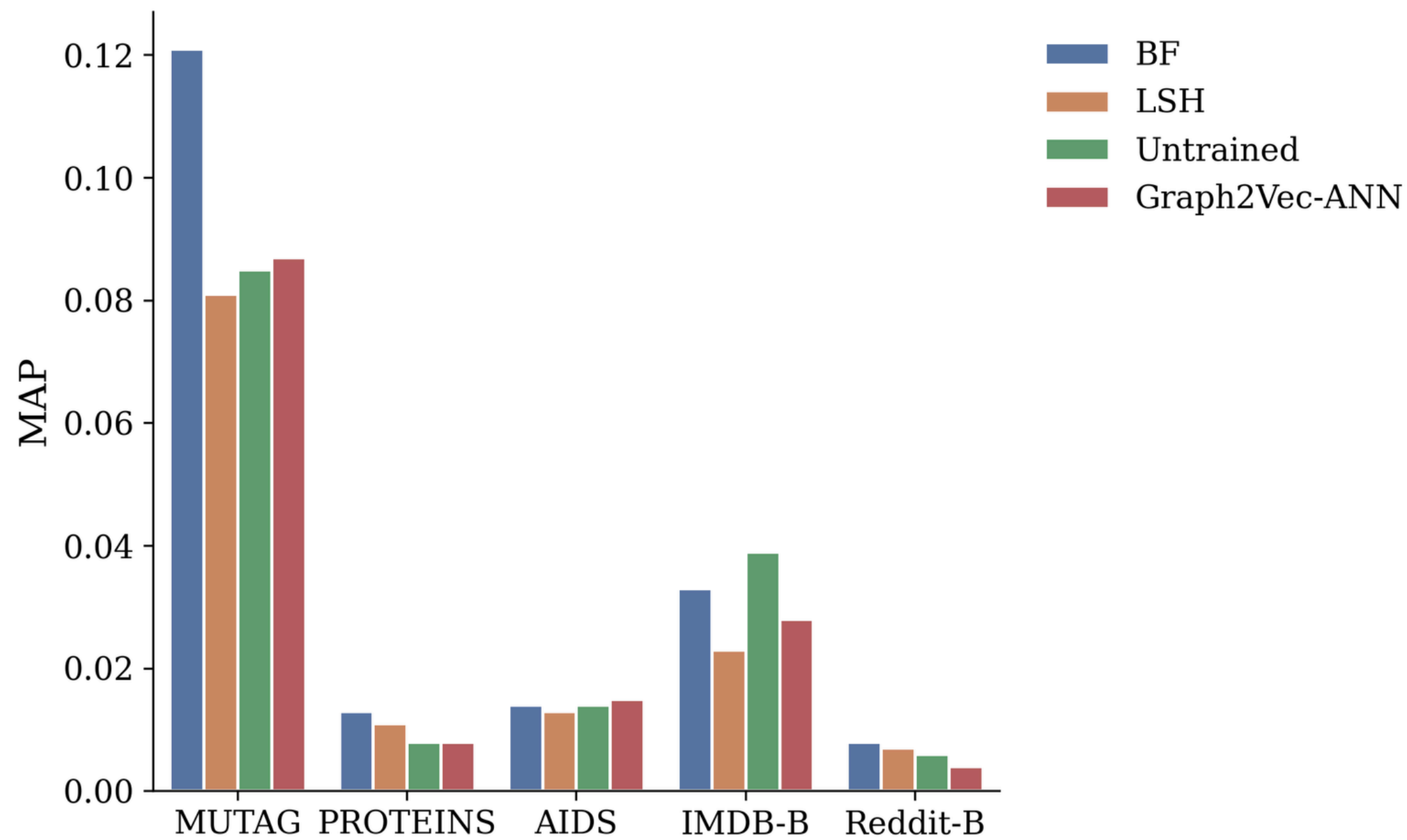
Precision@10 Across Datasets



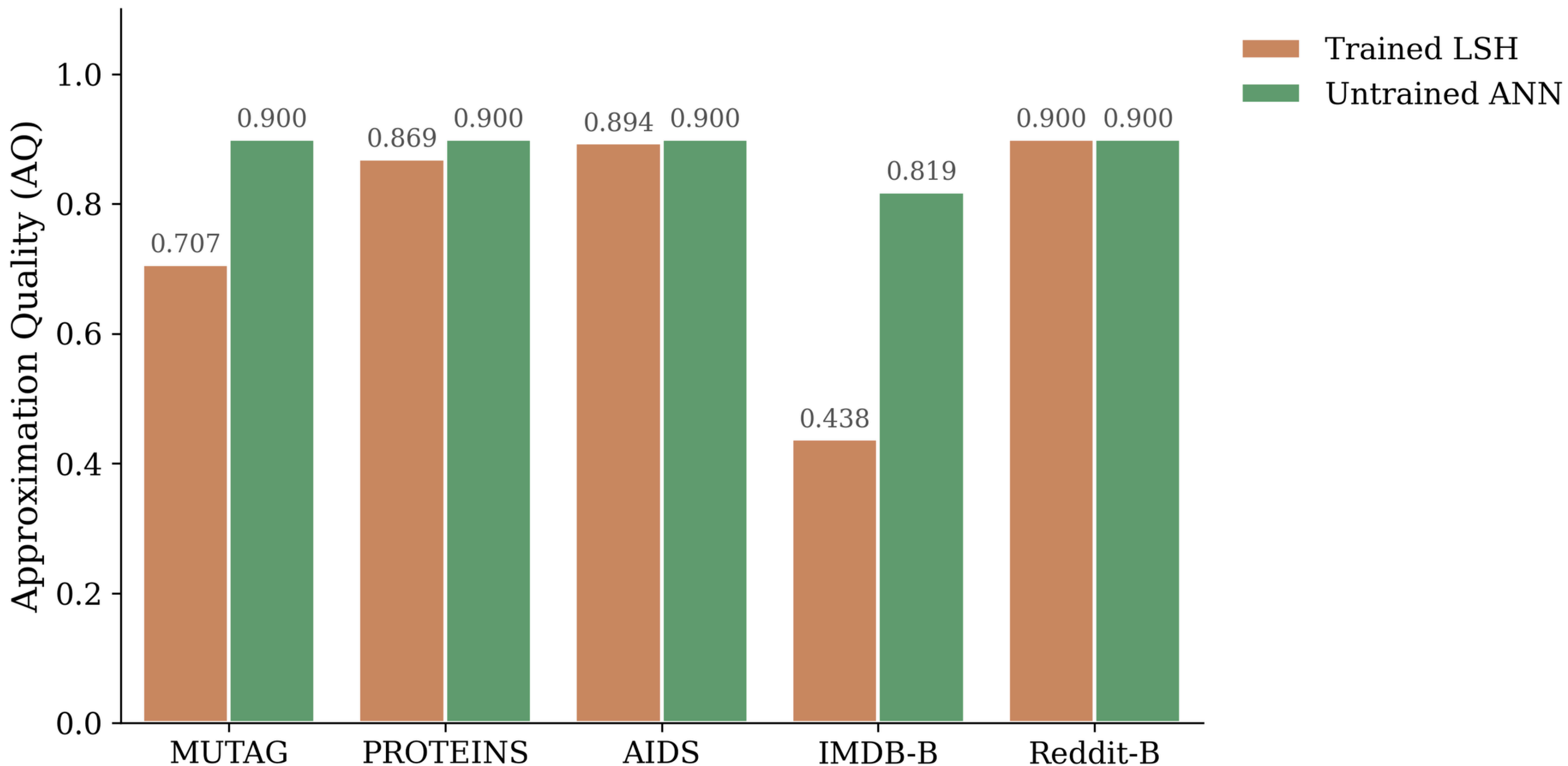
Recall@10



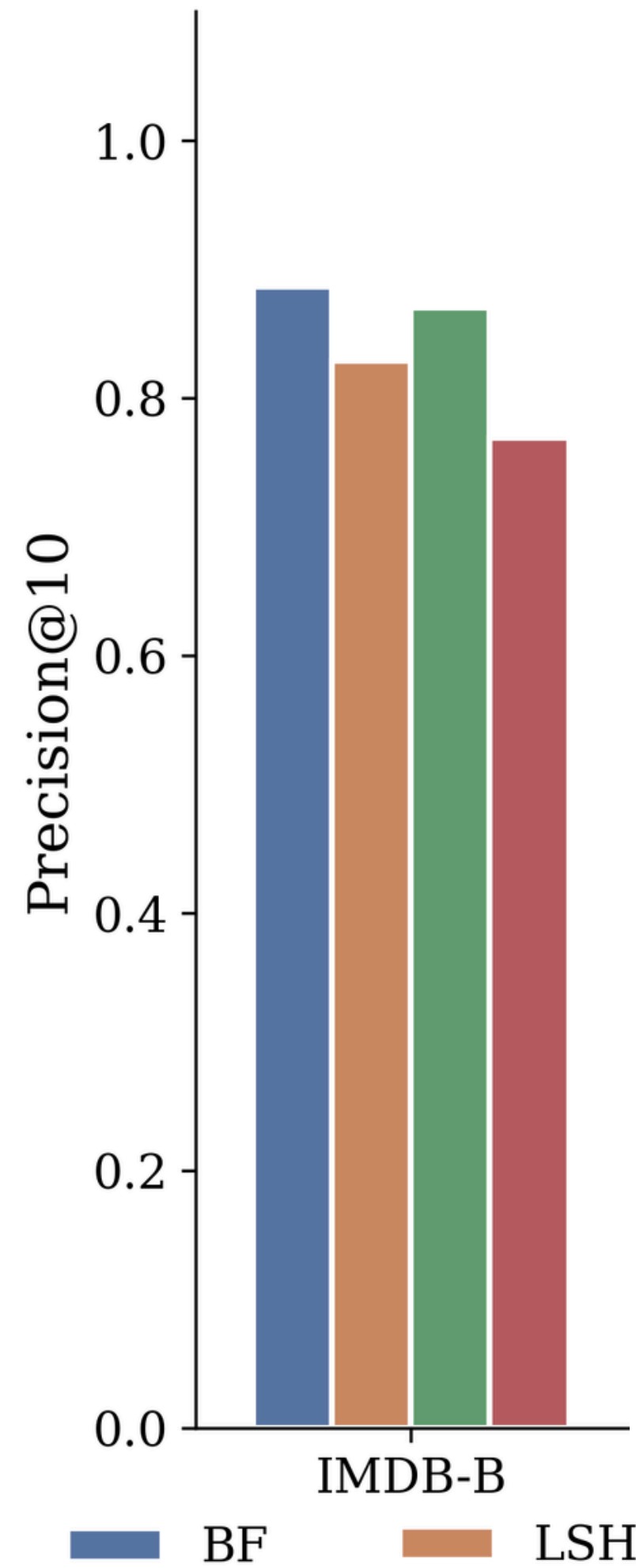
Mean Average Precision (MAP)



Approximation Quality (AQ) Comparison



IMDB-Binary Anomaly



Observation

- Untrained LSH performs better than trained LSH
- But under Brute-Force, trained embeddings are better

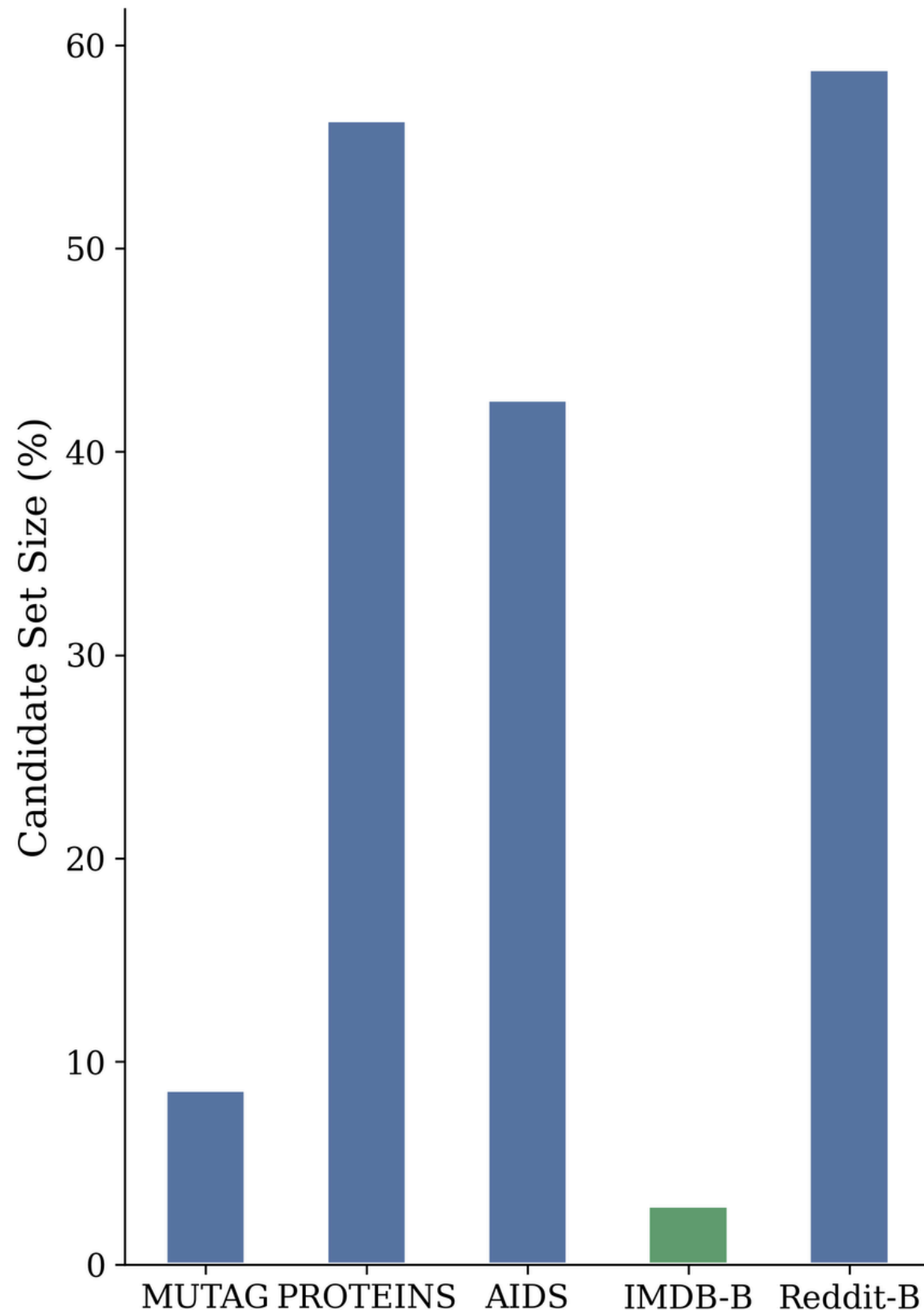
Contradiction

- Training improves embeddings
- Yet retrieval performance drops with LSH

Trained LSH: AQ $\approx$ 0.44
Untrained LSH: AQ $\approx$ 0.82

The issue is not embedding quality, but retrieval mechanism

IMDB-Binary Anomaly



Key Problem

- LSH retrieves only $\sim 2.9\%$ of corpus for IMDB
- Other datasets: $\sim 40\text{--}60\%$

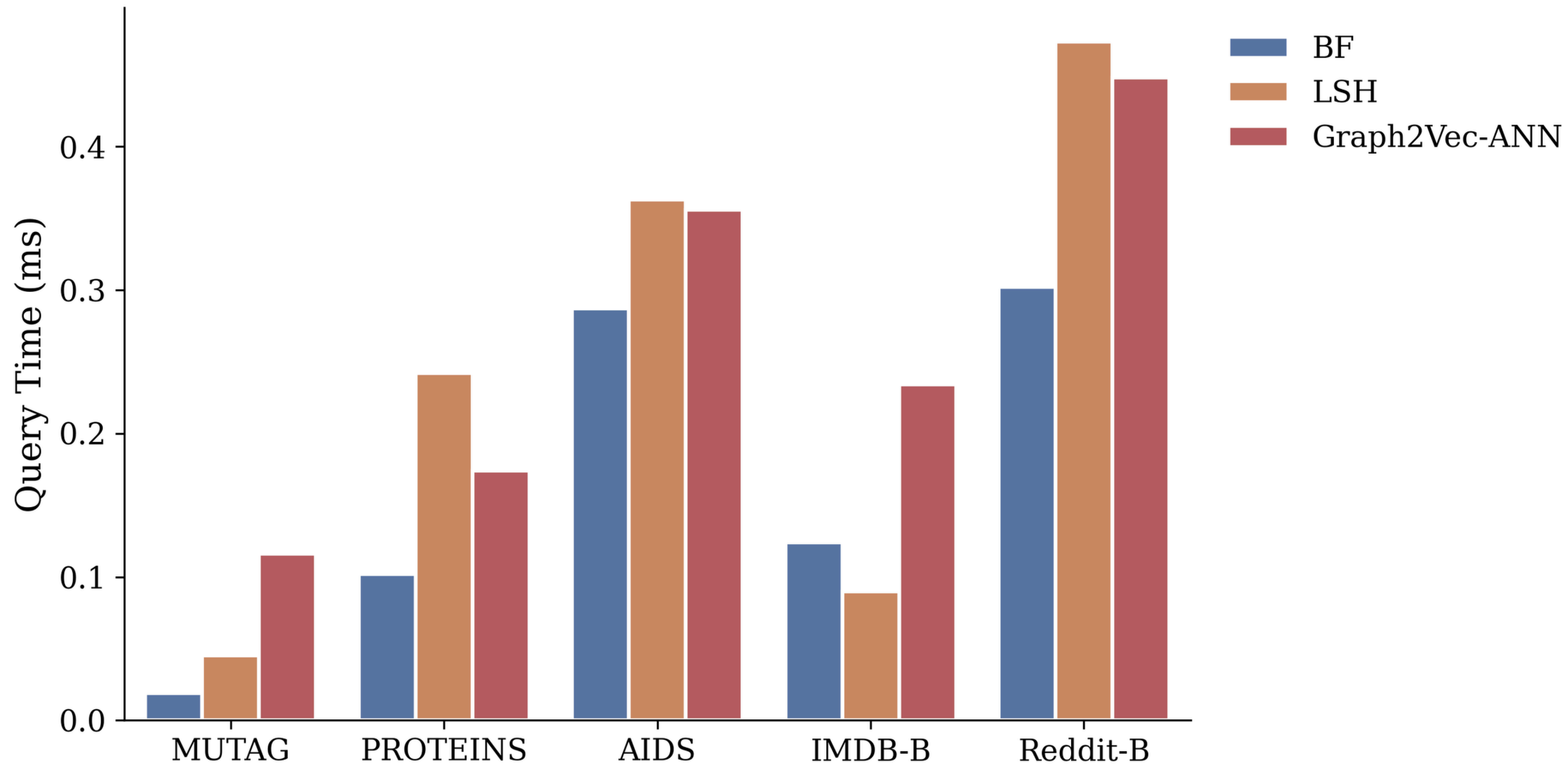
Why it happens

- No node features $\rightarrow$ weak signal
- Embeddings become poorly distributed for LSH
- Hash buckets fail to capture neighbors

True nearest neighbors are never retrieved $\rightarrow$ re-ranking fails

- LSH fails due to poor bucket coverage
- Better indexing (e.g., HNSW) would fix this

Average Query Time (ms) Comparison



Limitations

1. GED Supervision Noise

- Training uses approximate GED (beam search)
- Introduces label noise near decision boundary

2. LSH Sensitivity

- Performance depends on embedding distribution
- Can fail when bucket coverage is poor (IMDB)

3. Normalized Mutual Information

- Feature-poor datasets (e.g., IMDB) are harder
- Degree-based features provide weak signals

4. Scale Limitation

- Experiments limited to small datasets (≤ 2000 graphs)
- Cannot fully validate LSH speedup.

5. Model Design Constraints

- No hard negative mining
- Fixed architecture (no hyperparameter tuning)

References

- [1.] Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How powerful are graph neural networks? International Conference on Learning Representations (ICLR).
- [2.] Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. Proceedings of CVPR.
- [3.] Datar, M., Immorlica, N., Indyk, P., & Mirrokni, V. (2004). Locality-sensitive hashing scheme based on p-stable distributions. Proceedings of SoCG.
- [4.] Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: Towards removing the curse of dimensionality. Proceedings of STOC.
- [5.] Bai, Y., Ding, H., Gu, K., Sun, Y., & Wang, W. (2019). SimGNN: A neural network approach to fast graph similarity computation. Proceedings of WSDM.
- [6.] Narayanan, A., Chandramohan, M., Venkatesan, R., Chen, L., Liu, Y., & Jaiswal, S. (2017). Graph2Vec: Learning distributed representations of graphs. arXiv preprint arXiv:1707.05005.